

第09章：上下文与记忆

讲师：尚硅谷-宋红康

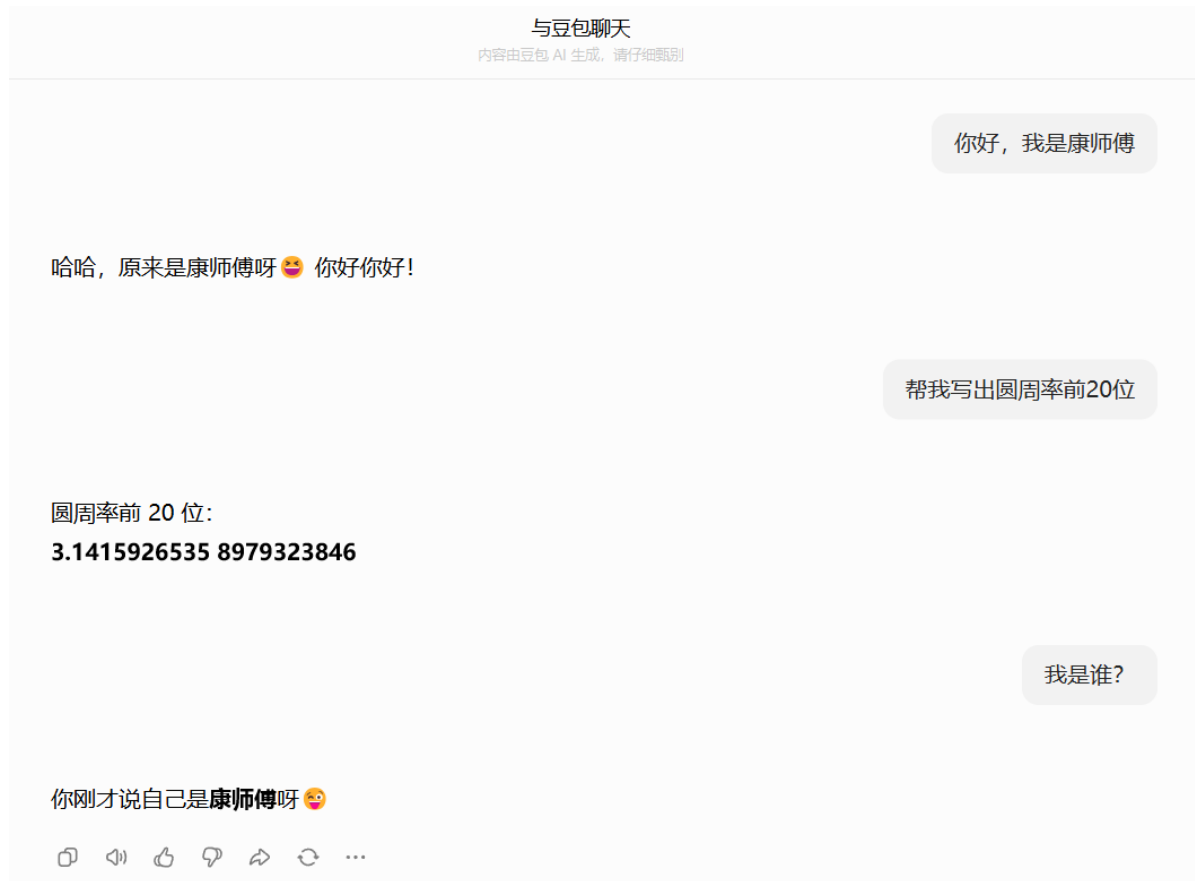
官网：[尚硅谷](#)

1、概述

1.1 为什么需要记忆(Memory)

记忆是一种记住之前互动信息的系统。随着Agent处理涉及大量用户交互的复杂任务，记忆变得至关重要！

大多数的大模型应用程序都会有一个 **会话接口**，允许我们进行 **多轮的对话**，并有一定的上下文记忆能力。比如：



但实际上，大模型本身是“无状态”的，**不会记忆**任何上下文的。即每次调用 `agent.invoke()` 都是全新的开始，不记得之前的对话。

举例：

```

1 from langchain.chat_models import init_chat_model
2 from dotenv import load_dotenv
3 import os
4
5 # 从.env文件中加载环境变量
6 load_dotenv(override=True)
7
8 model = init_chat_model(
9     model="gpt-5.4-mini",
10    model_provider="openai",
11    api_key=os.getenv("CLOSEAI_API_KEY"),
12    base_url=os.getenv("CLOSEAI_BASE_URL")
13 )

```

情况1:

```

1 from langchain.agents import create_agent
2 from langchain_core.messages import HumanMessage, AIMessage
3
4 messages1 = [
5     HumanMessage("你好，我叫小明"),
6     AIMessage("很高兴认识你，小明"),
7     HumanMessage("你用一句话介绍下你自己"),
8 ]
9
10 agent = create_agent(
11     model=model,
12     tools=[],
13 )
14
15 response1 = agent.invoke({"messages" : messages1})
16
17 for msg in response1["messages"]:
18     msg.pretty_print()

```

```

1  ===== Human Message
2  =====
3  你好，我叫小明
4  ===== Ai Message
5  =====
6  很高兴认识你，小明
7  ===== Human Message
8  =====
9  你用一句话介绍下你自己
10 ===== Ai Message
11 =====
12 我是一个可以帮助你回答问题、写作、翻译和思考的 AI 助手。

```

```

1 messages2 = [HumanMessage("我叫什么名字? ")]
2
3 response2 = agent.invoke({"messages" : messages2})
4
5 for msg in response2["messages"]:
6     msg.pretty_print()

```

```

1 ===== Human Message
2 =====
3 我叫什么名字?
4 ===== Ai Message
5 =====
6 我不知道你的名字，除非你告诉我。
7 如果你愿意，可以直接告诉我，我就记住并称呼你。

```

情况2:

```

1 messages3 = [
2     HumanMessage("你好，我叫小明"),
3     AIMessage("很高兴认识你，小明"),
4     HumanMessage("你用一句话介绍下你自己"),
5     AIMessage("我是一个由 OpenAI 训练的人工智能助手，可以帮你回答问题、写作和解决各种任
6     务。"),
7     HumanMessage("我叫什么名字? ")
8 ]
9 response3 = agent.invoke({"messages" : messages3})
10
11 for msg in response3["messages"]:
12     msg.pretty_print()

```

```

1 ===== Human Message
2 =====
3 你好，我叫小明
4 ===== Ai Message
5 =====
6 很高兴认识你，小明
7 ===== Human Message
8 =====
9 你用一句话介绍下你自己
10 ===== Ai Message
11 =====
12 我是一个由 OpenAI 训练的人工智能助手，可以帮你回答问题、写作和解决各种任务。
13 ===== Human Message
14 =====
15 我叫什么名字?
16 ===== Ai Message
17 =====

```

我们希望的智能体应该是右图的情况：



1.2 如何解决记忆问题

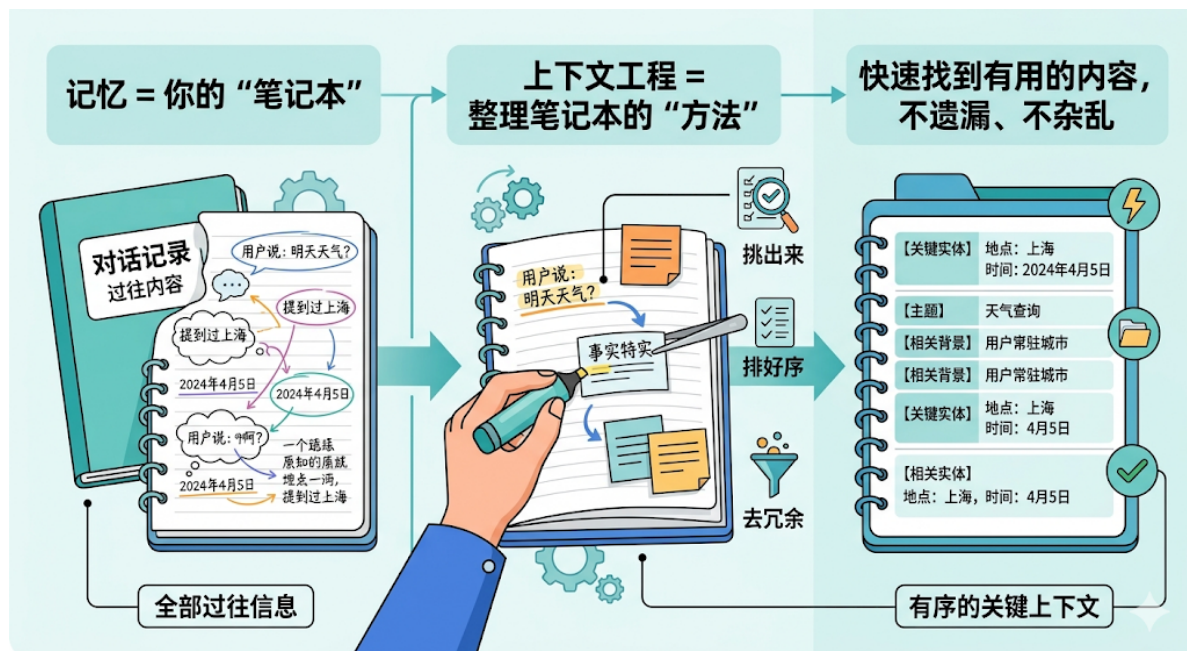
1.2.1 上下文工程

实现这个记忆功能，就需要 **额外的模块** 去保存我们和模型对话的上下文信息，然后在下一次请求时，把历史信息都输入给模型，让模型输出结果。

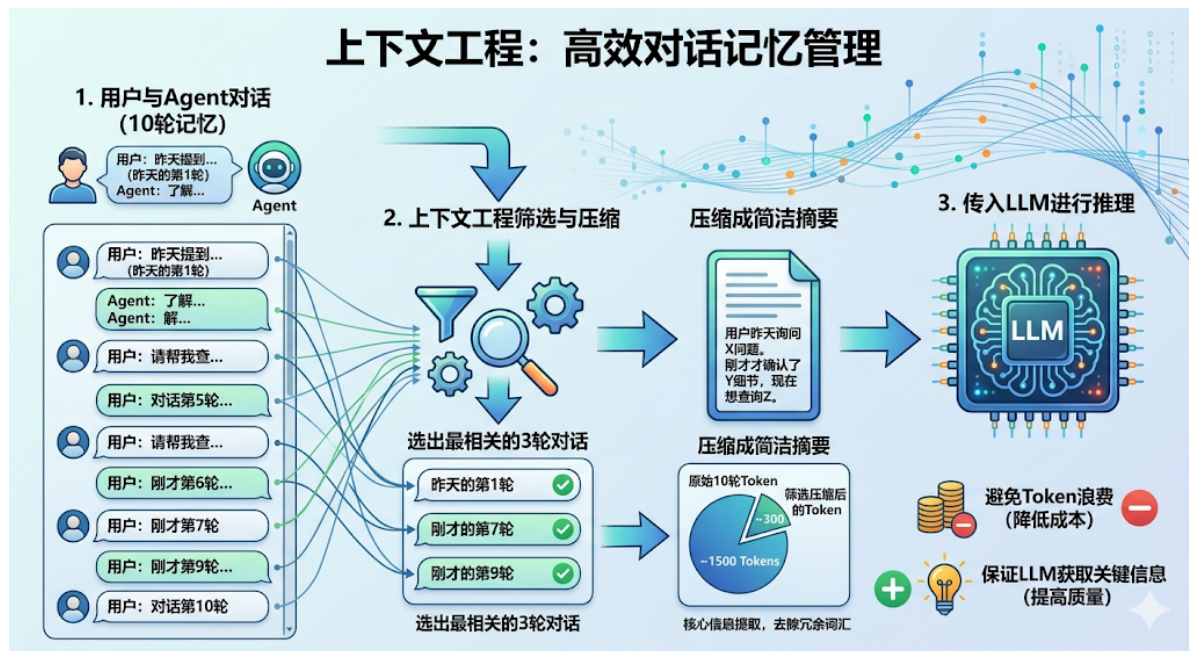
在 LangChain 中，**记忆(Memory)** 就是专门负责“**存储历史交互信息**”的组件，核心作用是「**保存上下文**」和「**提供上下文**」，让LLM在每次响应时，都能“看到”之前的对话内容。

上下文工程(Context Engineering) 负责“**合理组织**”这些记忆和任务信息，让LLM的响应更连贯、更贴合需求。这也是Agent能实现 **复杂多轮交互** 的核心基础。

类比：



举例：



1.2.2 上下文类型及相关的API

LangChain的上下文工程是基于Agent讨论的，而上下文工程是构建在LangGraph之上的。

LangGraph 提供了三种管理上下文的方法，这些方法结合了可变性和生命周期维度：

上下文类型	描述	可 变 性	生命 周期	访问方法
动态运行时上下文	在单次运行中会演变的可变数据	动态	单次运行	LangGraph state 对象
动态跨会话上下文	在对话间共享的持久数据。比如用户偏好、历史洞察、知识条目	动态	跨对话	LangGraph store 对象
静态运行时上下文	在启动时传入的用户元数据、工具、数据库连接	静态	单次运行	LangGraph context 对象

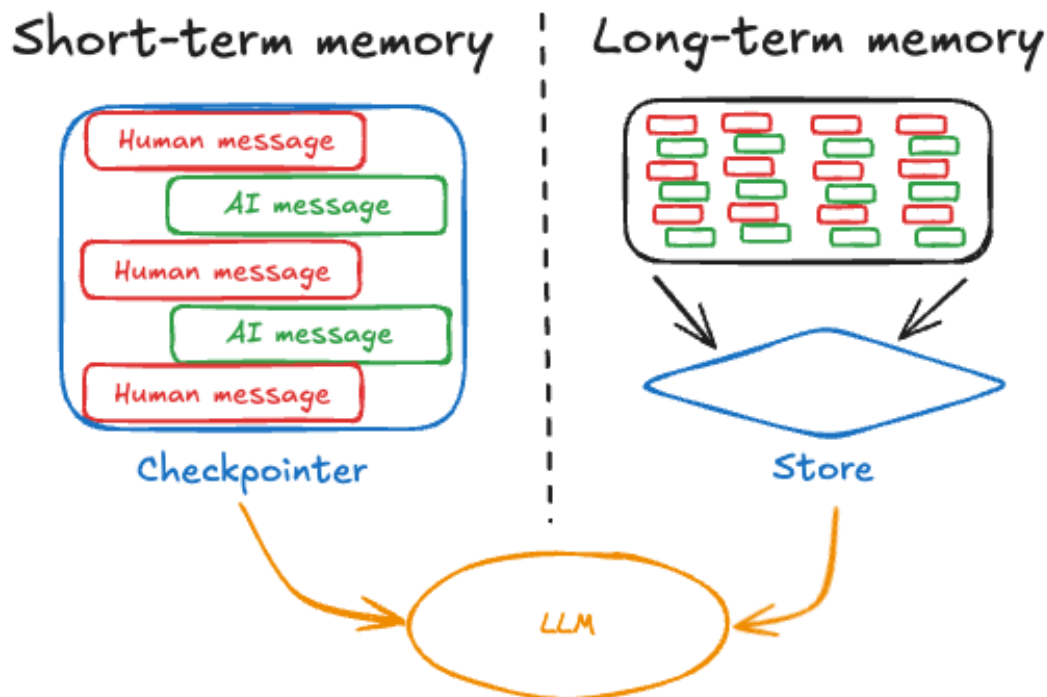
1.3 LangChain的记忆

1.3.1 记忆的分类

官方说明：<https://docs.langchain.com/oss/python/concepts/memory>

记忆分为短期记忆和长期记忆，对应不同的使用场景：

- **短期记忆 (Short-term memory、会话级记忆、thread-scoped memory)**：作用范围是单个对话线程 (Thread) 内，一旦开启新对话 (更换 `thread_id`)，记忆即消失。
- **长期记忆 (Long-term memory，跨会话级记忆)**：在会话间存储用户特定或应用级数据，并在 `会话线程间共享`。它可以随时在任何线程中被调用。记忆的范围是任意自定义命名空间，而不仅仅是单一线程 ID。



1.3.2 记忆的管理

在LangChain v0.x版本中，通过专用的xxxMemory类管理记忆。

在LangChain v1.x版本中，Agent是构建在LangGraph图结构之上的，通过上文提到的state和store构建记忆系统。使用更简单、功能更统一。

- **state**：短期记忆对象，以 [会话](#) 为单位组织，包含当前会话的所有消息记录以及自定义信息。
- **store**：长期记忆对象，[跨会话持久化](#) 的数据，通常需结合向量数据库或外部存储实现。

2、短期记忆

LangChain1.x 的短期记忆是三者的组合：

State（会话内部状态） + Checkpointer（持久化机制） + Thread ID（会话作用域）

- **State**：默认 [存储历史消息列表messages](#)，通过State 管理历史消息
- **Checkpointer**：负责将State 作为检查点持久化保存，检查点是某个时刻的State 快照
- **Thread ID**：用于唯一标识State，LangChain运行时会按照 thread_id 读写State快照

这就像玩 RPG 游戏时的“自动存档”：你不需要手动保存，系统在关键节点自动记录，下次进入游戏随时可以从上次的存档点继续。

2.1 基于内存的持久化器

这是最便捷的使用方式，适合快速测试或调试。

2.1.1 举例1：没有记忆

```
1 from langchain.agents import create_agent
2 from langchain.messages import HumanMessage
3
4 agent = create_agent(
5     model=model,
6     tools=[]
7 )
```

```

8
9 print("\n第一轮对话: ")
10 response1 = agent.invoke({
11     "messages": [HumanMessage("我叫张三")]
12 })
13 print(f"Agent: {response1['messages'][-1].content}")
14
15 print("\n第二轮对话: ")
16 response2 = agent.invoke({
17     "messages": [HumanMessage("我叫什么? ")]
18 })
19 print(f"Agent: {response2['messages'][-1].content}")

```

```

1 第一轮对话:
2  Agent: 你好, 张三! 很高兴认识你。有什么我可以帮你的吗?
3
4 第二轮对话:
5  Agent: 我不知道你的名字, 除非你告诉我。
6  如果你愿意, 可以直接发给我, 我就能记住在这次对话里称呼你。

```

2.1.2 举例2: 拥有记忆

步骤1: 第一二轮对话

```

1  from langgraph.checkpoint.memory import InMemorySaver
2
3  checkpointer = InMemorySaver()
4
5  # 1. 创建 Agent 时添加 checkpointer
6  agent = create_agent(
7      model=model,
8      checkpointer=checkpointer # 添加内存管理
9  )
10
11 # 2. 调用时指定 thread_id
12 config = {
13     "configurable": {
14         "thread_id": "1"
15     }
16 }
17
18 print("\n第一轮对话: ")
19 response1 = agent.invoke({
20     "messages": [HumanMessage("我叫张三")]},
21     config=config # 传入 config
22 )
23 print(f"Agent: {response1['messages'][-1].content}")
24
25 print("\n第二轮对话: ")
26 response2 = agent.invoke({
27     "messages": [HumanMessage("我叫什么? ")]},
28     config=config # 使用相同的 thread_id
29 )
30 print(f"Agent: {response2['messages'][-1].content}")

```

```
1 第一轮对话:
2  Agent: 你好, 张三! 很高兴认识你。有什么我可以帮你的吗?
3
4 第二轮对话:
5  Agent: 你叫张三。
```

说明: 你只需传入 `checkpointer` 和 `config`, `Agent` 就能自然具备连续对话能力。

```
1  from rich import print as rprint
2
3  latest_state = agent.get_state(config)
4  rprint(latest_state)
```

```
1  StateSnapshot(
2      values={
3          'messages': [
4              HumanMessage(
5                  content='我叫张三',
6                  additional_kwargs={},
7                  response_metadata={},
8                  id='16672c9b-4fac-44ca-830d-12908bad2c59'
9              ),
10             AIMessage(
11                 content='你好, 张三! 很高兴认识你。有什么我可以帮你的吗?',
12                 additional_kwargs={'refusal': None},
13                 response_metadata={
14                     'token_usage': {
15                         'completion_tokens': 22,
16                         'prompt_tokens': 10,
17                         'total_tokens': 32,
18                         'completion_tokens_details': {
19                             'accepted_prediction_tokens': 0,
20                             'audio_tokens': 0,
21                             'reasoning_tokens': 0,
22                             'rejected_prediction_tokens': 0
23                         },
24                         'prompt_tokens_details': {'audio_tokens': 0,
25 'cached_tokens': 0},
26                     'latency_checkpoint': {
27                         'engine_tbt_ms': 5,
28                         'engine_ttft_ms': 52,
29                         'engine_ttt_ms': 155,
30                         'pre_inference_ms': 80,
31                         'service_tbt_ms': 5,
32                         'service_ttft_ms': 696,
33                         'service_ttt_ms': 792,
34                         'total_duration_ms': 721,
35                         'user_visible_ttft_ms': 616
36                     }
37                 },
38                 'model_provider': 'openai',
39                 'model_name': 'gpt-5.4-mini-2026-03-17',
40                 'system_fingerprint': None,
41                 'id': 'chatcmp1-DpDsXsSSbHRVHJlJ42b19uMDGKRGQ',
42                 'service_tier': 'default',
43                 'finish_reason': 'stop',
```



```

43         'logprobs': None
44     },
45     id='lc_run--019eb1db-f785-79a1-b52a-d3b718e5b1da-0',
46     tool_calls=[],
47     invalid_tool_calls=[],
48     usage_metadata={
49         'input_tokens': 10,
50         'output_tokens': 22,
51         'total_tokens': 32,
52         'input_token_details': {'audio': 0, 'cache_read': 0},
53         'output_token_details': {'audio': 0, 'reasoning': 0}
54     }
55 ),
56 HumanMessage(
57     content='我叫什么? ',
58     additional_kwargs={},
59     response_metadata={},
60     id='ea385d96-ba1e-4b40-9b53-9e2848f751f0'
61 ),
62 AIMessage(
63     content='你叫张三。',
64     additional_kwargs={'refusal': None},
65     response_metadata={
66         'token_usage': {
67             'completion_tokens': 9,
68             'prompt_tokens': 41,
69             'total_tokens': 50,
70             'completion_tokens_details': {
71                 'accepted_prediction_tokens': 0,
72                 'audio_tokens': 0,
73                 'reasoning_tokens': 0,
74                 'rejected_prediction_tokens': 0
75             },
76             'prompt_tokens_details': {'audio_tokens': 0,
77 'cached_tokens': 0},
78             'latency_checkpoint': {
79                 'engine_tbt_ms': 3,
80                 'engine_ttft_ms': 38,
81                 'engine_ttt_ms': 64,
82                 'pre_inference_ms': 96,
83                 'service_tbt_ms': 4,
84                 'service_ttft_ms': 295,
85                 'service_ttt_ms': 310,
86                 'total_duration_ms': 227,
87                 'user_visible_ttft_ms': 199
88             }
89         },
90         'model_provider': 'openai',
91         'model_name': 'gpt-5.4-mini-2026-03-17',
92         'system_fingerprint': None,
93         'id': 'chatcpl-DpDsYCd016maB6yakkiFJknskxd0C',
94         'service_tier': 'default',
95         'finish_reason': 'stop',
96         'logprobs': None
97     },
98     id='lc_run--019eb1dc-0139-7cc3-bd9a-1d608b2fd48f-0',
99     tool_calls=[],
100     invalid_tool_calls=[],

```

```

100         usage_metadata={
101             'input_tokens': 41,
102             'output_tokens': 9,
103             'total_tokens': 50,
104             'input_token_details': {'audio': 0, 'cache_read': 0},
105             'output_token_details': {'audio': 0, 'reasoning': 0}
106         }
107     )
108 ]
109 },
110 next=(),
111 config={
112     'configurable': {
113         'thread_id': '1',
114         'checkpoint_ns': '',
115         'checkpoint_id': '1f164d5b-625c-679d-8004-1ef8a3347091'
116     }
117 },
118 metadata={'source': 'loop', 'step': 4, 'parents': {}},
119 created_at='2026-06-10T14:07:27.193684+00:00',
120 parent_config={
121     'configurable': {
122         'thread_id': '1',
123         'checkpoint_ns': '',
124         'checkpoint_id': '1f164d5b-570f-6b8f-8003-88f1894a5042'
125     }
126 },
127 tasks=(),
128 interrupts=()
129 )

```

步骤2：第三轮对话

当我们再次进行对话时，直接带入线程ID，即可带入此前对话记忆：

```

1 print("\n第三轮对话：")
2 response3 = agent.invoke({
3     "messages": [HumanMessage("我刚才问了什么问题? ") ]},
4     config=config # 使用相同的 thread_id
5 )
6 print(f"Agent: {response3['messages'][-1].content}")

```

```

1 第三轮对话：
2 Agent: 你刚才问的是：“我叫什么？”

```

```

1 from rich import print as rprint
2
3 latest_state = agent.get_state(config)
4 rprint(latest_state)

```

输出：略

说明：第三轮对话中的 `"我刚才问了什么问题？"` 就能在第二轮存储的消息历史中找到答案。所有消息历史都会自动追加到 `AgentState` 的 `messages` 字段中，无需手动维护。

步骤3：更新线程ID

而如果更新线程ID，则会重新开启对话：

```
1 config2 = {
2     "configurable": {
3         "thread_id": "2"
4     }
5 }
6
7 response4 = agent.invoke(
8     {"messages": [HumanMessage("你还记得我叫什么名字么? ") ]},
9     config=config2
10 )
11
12 print(response4['messages'][-1].content)
```

```
1 我目前**不知道你的名字**，因为你还没有告诉我。
2 如果你愿意，可以直接发我你的名字，我就用这个称呼你。
```

说明：thread_id 隔离不同会话空间。

2.1.3 关键步骤说明

第1步：初始化记忆引擎： `checkpointer = InMemorySaver()` ——创建一个内存级的记忆存储。

注意：InMemorySaver内存中保存，进程结束就丢失数据，适合测试。生产环境可换成数据库持久化的 `SqliteSaver`、`PostgresSaver` 等

第2步：绑定 Agent： 在 `create_agent` 时传入 `checkpointer`，让 Agent 具备状态存储能力。

第3步：设定会话 ID： 通过 `config = {"configurable": {"thread_id": "1"}}` 为每次调用指定线程标识。同一个 thread_id 共享记忆，不同 thread_id 完全隔离。

```
1 # 会话 1
2 config1 = {"configurable": {"thread_id": "1"}}
3 agent.invoke({...}, config=config1)
4
5 # 会话 2
6 config2 = {"configurable": {"thread_id": "2"}}
7 agent.invoke({...}, config=config2)
8
9 # 两个会话完全独立
```

`thread_id` 是记忆管理的核心开关：在会话2里询问会话1的会话信息，Agent 会表示不知道——因为双方记忆空间完全隔离。

生产环境中：

场景1：多用户聊天

不同 thread_id = 不同会话，Agent 能正确记住每个会话的内容。

```
1 agent = create_agent(
2     model=model,
3     tools=[],
4     checkpointer=InMemorySaver()
5 )
6
```

```

7  # 用户 Alice
8  config_alice = {"configurable": {"thread_id": "user_alice"}}
9  agent.invoke({"messages": [...]}, config_alice)
10 ...
11
12
13 # 用户 Bob
14 config_bob = {"configurable": {"thread_id": "user_bob"}}
15 agent.invoke({"messages": [...]}, config_bob)
16 ...
17
18 # 两个会话完全独立

```

场景2：同一用户的不同任务

```

1  # 任务 1: 写代码
2  config_task1 = {"configurable": {"thread_id": "task_coding"}}
3  agent.invoke({"messages": [...]}, config_task1)
4  ...
5
6
7  # 任务 2: 写文档
8  config_task2 = {"configurable": {"thread_id": "task_docs"}}
9  agent.invoke({"messages": [...]}, config_task2)
10 ...

```

2.1.4 工作原理

内存保存了什么？

```

1  agent.invoke({"messages": [{"role": "user", "content": "你好"}]}, config)
2  # InMemorySaver 保存:
3  # {
4  #     "thread_id": "xxx",
5  #     "messages": [
6  #         HumanMessage("你好"),
7  #         AIMessage("你好！有什么可以帮助你吗？")
8  #     ]
9  # }
10
11 agent.invoke({"messages": [{"role": "user", "content": "天气"}]}, config)
12 # InMemorySaver 更新:
13 # {
14 #     "thread_id": "xxx",
15 #     "messages": [
16 #         HumanMessage("你好"),
17 #         AIMessage("你好！有什么可以帮助你吗？"),
18 #         HumanMessage("天气"),
19 #         AIMessage("...")
20 #     ]
21 # }

```

自动追加历史

```

1 # 你只需要传新消息
2 agent.invoke(
3     {"messages": [{"role": "user", "content": "新问题"}]},
4     config
5 )

```

此时，checkpointer 自动：

- ① 读取之前的历史
- ② 追加新消息
- ③ 调用模型（传入完整历史）
- ④ 保存新的历史

说明： checkpointer 会自动管理历史

2.1.5 常见问题

1、为什么 Agent 不记得？

检查：

- ☒ 是否添加了 `checkpointer=InMemorySaver()` ？
- ☒ 是否传入了 `config` 参数？
- ☒ 两次调用的 `thread_id` 是否相同？

```

1 # ✗ 错误：没有 checkpointer
2 agent = create_agent(model=model, tools=[])
3 agent.invoke({...}) # 不会记住
4
5
6 # ✗ 错误：没有 config
7 agent = create_agent(model=model, tools=[], checkpointer=InMemorySaver())
8 agent.invoke({...}) # 不会记住
9
10
11 # ✗ 错误：thread_id 不同
12 agent.invoke(..., config={"configurable": {"thread_id": "1"}})
13 agent.invoke(..., config={"configurable": {"thread_id": "2"}}) # 不同会话
14
15
16 # ☑ 正确
17 agent = create_agent(model=model, tools=[], checkpointer=InMemorySaver())
18 config = {"configurable": {"thread_id": "1"}}
19 agent.invoke(..., config)
20 agent.invoke(..., config) # 记得！

```

2、InMemorySaver 会丢失数据吗？

会！InMemorySaver 只保存在内存中：

- ☒ 同一进程内有效（不支持跨进程共享）
- ✗ 程序重启后丢失（或进程重启后丢失）
- ✗ 不同进程无法共享

解决方案：持久化（SQLite、PostgreSQL）

3、内存会无限增长吗？

会！默认情况下，InMemorySaver 会保存所有消息。

问题：

- 消息越来越多（无限增长，需要管理上下文）
- token消耗增加，甚至会超过模型的 token 限制
- 响应速度变慢、成本增加

解决方案：上下文管理（修剪、摘要）

4、如何清空某个会话的历史？

目前 InMemorySaver 没有提供删除 API。

临时方案：

- 使用新的 thread_id
- 或重新创建 Agent

2.2 基于外部存储介质的持久化器

如果将 状态检查点(checkpointer) 保存在内存， 进程结束 则状态丢失，生产环境不可接受。因此，生产环境要用持久化的外部存储介质，如PostgreSQL。LangGraph提供的checkpointer后端列表如下

<https://docs.langchain.com/oss/python/langgraph/persistence#checkpointer-libraries>

此处选择PostgreSQL作为持久化器。

2.2.1 数据库环境准备

在此之前，先准备好PostgreSQL环境，此处在云服务器的Ubuntu系统安装PostgreSQL。大家需要根据自己云服务器位置，更改URL中的IP即可。

具体云服务器安装和PostgreSQL的安装，见文件02-资料下的《Linux云服务器安装与PostgreSQL安装》

```
1 | URL:
   | postgresql://langchain_user:abcd1234@118.195.128.47:5432/langchain_db?
   | sslmode=disable
```

说明：

- 1、上述URL中的用户名、密码、IP地址，需要根据自己的情况替换。
- 2、要在LangChain中对接PostgreSQL，还需要额外的依赖，比如：

```
1 | pip install langgraph-checkpoint-postgres
```

其实我们在课程开始的requirements.txt中已经提供了相关依赖，此处**不必也不能重新安装**，否则版本可能冲突。

2.2.2 代码实现

```
1 from langchain.chat_models import init_chat_model
2 from dotenv import load_dotenv
3 import os
4
5 # 从.env文件中加载环境变量
6 load_dotenv(override=True)
7
8 model = init_chat_model(
9     model="gpt-5.4-mini",
10    model_provider="openai",
11    api_key=os.getenv("CLOSEAI_API_KEY"),
12    base_url=os.getenv("CLOSEAI_BASE_URL")
13 )
```

```
1 from langchain.agents import create_agent
2 from langchain.messages import HumanMessage
3 from langgraph.checkpoint.postgres import PostgresSaver
4
5
6 DB_URL =
7 "postgresql://langchain_user:abcd1234@118.195.128.47:5432/langchain_db?
8 sslmode=disable"
9
10 with PostgresSaver.from_conn_string(DB_URL) as checkpointer:
11     # 初始化PostgreSQL数据库
12     checkpointer.setup()
13
14     agent = create_agent(
15         model=model,
16         checkpointer=checkpointer
17     )
18
19     config = {"configurable": {"thread_id": "1"}}
20     response1 = agent.invoke(
21         {"messages": [HumanMessage("你好，我是老王")]},
22         config=config
23     )
24
25     print("=" * 30, "-> 第一次调用 <=", "=" * 30)
26     for msg in response1["messages"]:
27         msg.pretty_print()
28
29     response2 = agent.invoke(
30         {"messages": [HumanMessage("你好，我是谁? ")]},
31         config=config
32     )
33
34     print("=" * 30, "-> 第二次调用 <=", "=" * 30)
35     for msg in response2["messages"]:
36         msg.pretty_print()
```

输出

```
1 ===== -> 第一次调用 <=
=====
```

```

2  ===== Human Message
   =====
3
4  你好，我是老王
5  ===== Ai Message
   =====
6
7  你好，老王！很高兴认识你。有什么我可以帮你的？
8  ===== -> 第二次调用 <-
   =====
9  ===== Human Message
   =====
10
11 你好，我是老王
12 ===== Ai Message
   =====
13
14 你好，老王！很高兴认识你。有什么我可以帮你的？
15 ===== Human Message
   =====
16
17 你好，我是谁？
18 ===== Ai Message
   =====
19
20 你是老王。

```

`setup()` 用于初始化PostgreSQL数据库，首次运行会创建必要的表，重复执行不会重新建表，底层逻辑是 `Create IF Not Exists`，相关源码如下

```

1  def setup(self) -> None:
2      """Set up the checkpoint database asynchronously.
3
4      This method creates the necessary tables in the Postgres database if they
      don't already exist and runs database migrations. It MUST be called directly
      by the user the first time checkpoint is used.
5      """
6      ...

```

2.2.3 查看持久化数据

查看PostgreSQL数据库，可以通过命令行或图形化工具查看

```

1  ubuntu@vm-0-6-ubuntu:~$ psql
   "postgresql://langchain_user:abcd1234@localhost:5432/langchain_db?
   sslmode=disable"

```

```

1  psql (16.13 (Ubuntu 16.13-0ubuntu0.24.04.1))
2  Type "help" for help.
3
4  langgraph_db=>

```

PostgreSQL的存储结构是 `Database -> Schema -> Table`

- `Database`：数据库
- `Schema`：分区

- `Table` : 表

需求1: 查看所有数据库

```

1 langgraph_db=> \l
2                                     List of
3 databases
4   Name | Owner | Encoding | Locale Provider | Collate |
5   Ctype | ICU Locale | ICU Rules | Access privileges
6   -----+-----+-----+-----+-----+
7   langgraph_db | langgraph_user | UTF8 | libc | C.UTF-8 |
8   C.UTF-8 | | | =Tc/langgraph_user | +
9   | | | langgraph_user=CTc/langgraph_user
10  postgres | postgres | UTF8 | libc | C.UTF-8 |
11  C.UTF-8 | | |
12  template0 | postgres | UTF8 | libc | C.UTF-8 |
13  C.UTF-8 | | | =c/postgres | +
14  | | | postgres=CTc/postgres
15  template1 | postgres | UTF8 | libc | C.UTF-8 |
16  C.UTF-8 | | | =c/postgres | +
17  | | | postgres=CTc/postgres
18  (4 rows)

```

命令行前缀 `langgraph_db` 标识了当前所在数据库

需求2: 查看所有schema

```

1 langgraph_db=> \dn
2           List of schemas
3   Name | Owner
4   -----+-----
5   public | pg_database_owner
6   (1 row)

```

需求3: 查看当前所处的schema

```

1 langgraph_db=> select current_schema();
2   current_schema
3   -----
4   public
5   (1 row)

```

需求4: 查看当前schema下的所有表

```

1 langgraph_db=> \dt
2                               List of relations
3 Schema |          Name          | Type |   Owner
4 -----+-----+-----+-----
5 public | checkpoint_blobs       | table | langgraph_user
6 public | checkpoint_migrations | table | langgraph_user
7 public | checkpoint_writes      | table | langgraph_user
8 public | checkpoints            | table | langgraph_user
9 (4 rows)

```

这四张表都是 `setup()` 函数初始化时创建的。

- `checkpoints`：这是**主表**，存每个 thread 在某个时刻的 **checkpoint 快照**。
- `checkpoint_blobs`：这张表专门存不适合直接内联进 `checkpoints.checkpoint` 的较复杂 channel 值。
- `checkpoint_writes`：这张表存的是**中间写入 / pending writes**，不是最终完整 checkpoint。
- `checkpoint_migrations`：这张表不是业务数据表，而是**迁移版本表**。

2.3 对比两种方式

举例1：基于内存存储

```

1 from langchain.agents import create_agent
2 from langchain.messages import HumanMessage
3 from langgraph.checkpoint.memory import InMemorySaver
4
5 agent = create_agent(
6     model=model,
7     checkpointer=InMemorySaver()
8 )
9 config = {"configurable": {"thread_id": "1"}}
10
11
12 print("=" * 30, "-> 第一次调用 <-", "=" * 30)
13 response1 = agent.invoke(
14     {"messages": [HumanMessage("你好，我是谁? ") ]},
15     config=config
16 )
17 for msg in response1["messages"]:
18     msg.pretty_print()
19
20
21 print("=" * 30, "-> 第二次调用 <-", "=" * 30)
22 response2 = agent.invoke(
23     {"messages": [HumanMessage("我是老王") ]},
24     config=config
25 )
26 for msg in response2["messages"]:
27     msg.pretty_print()
28
29
30 print("=" * 30, "-> 第三次调用 <-", "=" * 30)
31 response3 = agent.invoke(
32     {"messages": [HumanMessage("你好，我是谁? ") ]},
33     config
34 )

```

```
35 for msg in response3["messages"]:
36     msg.pretty_print()
```

输出如下

```
1  ===== -> 第一次调用 <-
   =====
2  ===== Human Message
   =====
3
4  你好，我是谁？
5  ===== Ai Message
   =====
6
7  你好！我不知道你的真实身份，除非你告诉我。
8  如果你愿意，我可以根据你提供的信息，帮你判断你是谁、你像哪类人，或者帮你整理一段自我介绍。
9  ===== -> 第二次调用 <-
   =====
10 ===== Human Message
   =====
11
12 你好，我是谁？
13 ===== Ai Message
   =====
14
15 你好！我不知道你的真实身份，除非你告诉我。
16 如果你愿意，我可以根据你提供的信息，帮你判断你是谁、你像哪类人，或者帮你整理一段自我介绍。
17 ===== Human Message
   =====
18
19 我是老王
20 ===== Ai Message
   =====
21
22 你好，老王！很高兴认识你。
23 如果你愿意，我可以直接这样称呼你。
24 ===== -> 第三次调用 <-
   =====
25 ===== Human Message
   =====
26
27 你好，我是谁？
28 ===== Ai Message
   =====
29
30 你好！我不知道你的真实身份，除非你告诉我。
31 如果你愿意，我可以根据你提供的信息，帮你判断你是谁、你像哪类人，或者帮你整理一段自我介绍。
32 ===== Human Message
   =====
33
34 我是老王
35 ===== Ai Message
   =====
36
```

```

37 | 你好，老王！很高兴认识你。
38 | 如果你愿意，我可以直接这样称呼你。
39 | ===== Human Message
   | =====
40 |
41 | 你好，我是谁？
42 | ===== Ai Message
   | =====
43 |
44 | 你好，老王。你是老王。

```

可以发现，每次执行的输出完全相同，而我们并没有更改 `thread_id`，之所以看不到上次运行的状态是因为每次运行创建新的Saver()，历史State被丢弃了。

举例2：基于外部存储器存储

```

1 | from langchain.agents import create_agent
2 | from langchain.messages import HumanMessage
3 | from langgraph.checkpoint.postgres import PostgresSaver
4 |
5 | DB_URL =
   | "postgresql://langchain_user:abcd1234@118.195.128.47:5432/langchain_db?
   | sslmode=disable"
6 | with PostgresSaver.from_conn_string(DB_URL) as checkpointer:
7 |     # 初始化数据库
8 |     checkpointer.setup()
9 |
10 |    agent = create_agent(
11 |        model=model,
12 |        checkpointer=checkpointer
13 |    )
14 |
15 |    config = {"configurable": {"thread_id": "3"}}
16 |
17 |    print("=" * 30, "-> 第一次调用 <-", "=" * 30)
18 |    response1 = agent.invoke(
19 |        {"messages": [HumanMessage("你好，我是谁啊? ") ]},
20 |        config
21 |    )
22 |    for msg in response1["messages"]:
23 |        msg.pretty_print()
24 |
25 |
26 |    print("=" * 30, "-> 第二次调用 <-", "=" * 30)
27 |    response2 = agent.invoke(
28 |        {"messages": [HumanMessage("我是老王~") ]},
29 |        config
30 |    )
31 |    for msg in response2["messages"]:
32 |        msg.pretty_print()
33 |
34 |
35 |    print("=" * 30, "-> 第三次调用 <-", "=" * 30)
36 |    response3 = agent.invoke(
37 |        {"messages": [HumanMessage("你好，我是谁? ? ") ]},
38 |        config
39 |    )

```

```
40     for msg in response3["messages"]:
41         msg.pretty_print()
```

第一次输出：

```
1  ===== -> 第一次调用 <-
   =====
2  ===== Human Message
   =====
3
4  你好，我是谁啊？
5  ===== Ai Message
   =====
6
7  你好！我只能知道：你现在是这次和我对话的用户。
8  如果你是想问“我记不记得你是谁”，那我其实**不会自动知道你的真实身份**，除非你在对话里
   告诉我。
9
10 你可以直接告诉我你想让我怎么称呼你。
11 ===== -> 第二次调用 <-
   =====
12 ===== Human Message
   =====
13
14 你好，我是谁啊？
15 ===== Ai Message
   =====
16
17 你好！我只能知道：你现在是这次和我对话的用户。
18 如果你是想问“我记不记得你是谁”，那我其实**不会自动知道你的真实身份**，除非你在对话里
   告诉我。
19
20 你可以直接告诉我你想让我怎么称呼你。
21 ===== Human Message
   =====
22
23 我是老王~
24 ===== Ai Message
   =====
25
26 你好，老王~ 很高兴认识你！
27 接下来我可以怎么帮你？
28 ===== -> 第三次调用 <-
   =====
29 ===== Human Message
   =====
30
31 你好，我是谁啊？
32 ===== Ai Message
   =====
33
34 你好！我只能知道：你现在是这次和我对话的用户。
35 如果你是想问“我记不记得你是谁”，那我其实**不会自动知道你的真实身份**，除非你在对话里
   告诉我。
36
37 你可以直接告诉我你想让我怎么称呼你。
```



```

38  ===== Human Message
39  =====
40  我是老王~
41  ===== Ai Message
42  =====
43  你好，老王~ 很高兴认识你！
44  接下来我可以怎么帮你？
45  ===== Human Message
46  =====
47  你好，我是谁？？
48  ===== Ai Message
49  =====
50  你好，老王 🤖

```

第二次运行输出

```

1  ===== -> 第一次调用 <-
2  =====
3  ===== Human Message
4  =====
5  你好，我是谁啊？
6  ===== Ai Message
7  =====
8  你好！我只能知道：你现在是这次和我对话的用户。
9  如果你想问“我记不记得你是谁”，那我其实**不会自动知道你的真实身份**，除非你在对话
10  里告诉我。
11  ===== Human Message
12  =====
13  我是老王~
14  ===== Ai Message
15  =====
16  你好，老王~ 很高兴认识你！
17  接下来我可以怎么帮你？
18  ===== Human Message
19  =====
20  你好，我是谁？？
21  ===== Ai Message
22  =====
23  你好，老王 🤖
24  ===== Human Message
25  =====
26  你好，我是谁啊？
27  ===== Ai Message
28  =====

```

```
28
29 你好，老王。
30 ===== -> 第二次调用 <-
    =====
31 ===== Human Message
    =====
32
33 你好，我是谁啊？
34 ===== Ai Message
    =====
35
36 你好！我只能知道：你现在是这次和我对话的用户。
37 如果你是想问“我记不记得你是谁”，那我其实**不会自动知道你的真实身份**，除非你在对话
    里告诉我。
38
39 你可以直接告诉我你想让我怎么称呼你。
40 ===== Human Message
    =====
41
42 我是老王~
43 ===== Ai Message
    =====
44
45 你好，老王~ 很高兴认识你！
46 接下来我可以怎么帮你？
47 ===== Human Message
    =====
48
49 你好，我是谁？？
50 ===== Ai Message
    =====
51
52 你好，老王 🤖
53 ===== Human Message
    =====
54
55 你好，我是谁啊？
56 ===== Ai Message
    =====
57
58 你好，老王。
59 ===== Human Message
    =====
60
61 我是老王~
62 ===== Ai Message
    =====
63
64 你好，老王~ 有什么我可以帮你的？
65 ===== -> 第三次调用 <-
    =====
66 ===== Human Message
    =====
67
68 你好，我是谁啊？
69 ===== Ai Message
    =====
70
```

```

71 你好！我只能知道：你现在是这次和我对话的用户。
72 如果你想问“我记不记得你是谁”，那我其实**不会自动知道你的真实身份**，除非你在对话
    里告诉我。
73
74 你可以直接告诉我你想让我怎么称呼你。
75 ===== Human Message
    =====
76
77 我是老王~
78 ===== Ai Message
    =====
79
80 你好，老王~ 很高兴认识你！
81 接下来我可以怎么帮你？
82 ===== Human Message
    =====
83
84 你好，我是谁？？
85 ===== Ai Message
    =====
86
87 你好，老王 😊
88 ===== Human Message
    =====
89
90 你好，我是谁啊？
91 ===== Ai Message
    =====
92
93 你好，老王。
94 ===== Human Message
    =====
95
96 我是老王~
97 ===== Ai Message
    =====
98
99 你好，老王~ 有什么我可以帮你的？
100 ===== Human Message
    =====
101
102 你好，我是谁？？
103 ===== Ai Message
    =====
104
105 你好，老王。

```

根据输出判断，状态是累积的。

由此可以得出结论：即便重新创建 `Saver()`，只要 `thread_id` 一致，历史状态就可以和当前调用串联起来。

总结：

1. `InMemorySaver()`将状态持久化到内存，`进程结束或重建Saver()` 则历史状态丢失
2. 基于外部存储介质（如PostgreSQL）的持久化器，其存储的状态不会随进程终止而丢失，只要 **不显式删除历史状态**，即可通过 `thread_id` 加载历史状态。

2.4 记忆治理策略（上下文管理）

随着对话的进行，历史消息不断累积，`state`会持续增长，为模型带来挑战：

1. LLM的 `上下文窗口是有限的`，完整历史可能无法装入LLM的上下文窗口，导致上下文丢失或错误。
2. 即便模型的上下文窗口够大，多数LLM在长上下文场景仍然表现不佳。模型会被陈旧或离题的内容“分散注意力”。
3. 同时，会带来 `高昂的token花费`。

此时需要对上下文进行管理：对历史记录进行**压缩、清理、重组**等。

2.4.1 消息裁剪

调用模型前裁剪上下文。

目标是控制token用量，通常 `保留系统初始消息和最近若干消息`，或 `按token数保留末尾内容`。

适合成本敏感、对旧上下文依赖不强的场景。

```
1 from langchain.chat_models import init_chat_model
2 from dotenv import load_dotenv
3 import os
4
5 # 从.env文件中加载环境变量
6 load_dotenv(override=True)
7
8 model = init_chat_model(
9     model="gpt-5.4-mini",
10    model_provider="openai",
11    api_key=os.getenv("CLOSEAI_API_KEY"),
12    base_url=os.getenv("CLOSEAI_BASE_URL")
13 )
```

```
1 from langchain_core.messages import HumanMessage
2 from langchain.messages import RemoveMessage
3 from langgraph.graph.message import REMOVE_ALL_MESSAGES
4 from langgraph.checkpoint.memory import InMemorySaver
5 from langchain.agents import create_agent, AgentState
6 from langchain.agents.middleware import before_model
7 from langgraph.runtime import Runtime
8 from langchain_core.runnables import RunnableConfig
9 from typing import Any
10
11 @before_model
12 def trim_messages(state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
13     messages = state["messages"]
14
15     if len(messages) <= 3:
16         return None
17
18     first_msg = messages[0]
19     recent_messages = messages[-3:] if len(messages) % 2 == 0 else
messages[-4:]
20     new_messages = [first_msg] + recent_messages
21
```

```

22     return {
23         "messages": [
24             RemoveMessage(id=REMOVE_ALL_MESSAGES),
25             *new_messages
26         ]
27     }
28
29 agent = create_agent(
30     model=model,
31     middleware=[trim_messages],
32     checkpointer=InMemorySaver(),
33 )
34
35 config: RunnableConfig = {"configurable": {"thread_id": "1"}}
36
37 agent.invoke({"messages": [HumanMessage("你好，我是老王")]}), config)
38 agent.invoke({"messages": [HumanMessage("从现在起，你叫小王")]}), config)
39 agent.invoke({"messages": [HumanMessage("今天天气不错")]}), config)
40 final_response = agent.invoke({"messages": [HumanMessage("告诉我，你是谁？我是谁? ")]}, config)
41
42 for msg in final_response["messages"]:
43     msg.pretty_print()

```

假设有 5 条消息： [H1, A1, H2, A2, H3] （当前总数 5，奇数）。

执行 `messages[-4:]` → 取出 [A1, H2, A2, H3]。加上第一条 H1，最终大模型看到的是 [H1, A1, H2, A2, H3] （相当于没剪掉什么）。

输出

```

1  ===== Human Message
   =====
2
3  你好，我是老王
4  ===== Ai Message
   =====
5
6  好的，从现在起你可以叫我小王。
7  那我也叫你老王，可以吗？
8  ===== Human Message
   =====
9
10 今天天气不错
11 ===== Ai Message
   =====
12
13 是啊，天气不错的话，心情也容易好起来。
14 你今天有什么安排吗？
15 ===== Human Message
   =====
16
17 告诉我，你是谁？我是谁？
18 ===== Ai Message
   =====
19
20 我是你的AI助手。
21 你是老王。

```

显然，裁剪生效了。

分析：

我们来看看连续 4 次 `invoke` 时，内存里到底发生了什么：

步骤	触发动作 & 传入消息	触发前 State 中的 messages 列表 (长度)	trim_messages 是否触发与执行逻辑	最终传给 LLM 的 messages 实际内容
1	<code>invoke("你好，我是老王")</code>	<code>[H: 你好，我是老王]</code> (1)	不触发 (长度 ≤ 3)	<code>[H: 你好，我是老王]</code>
--	模型自动回复 (A1)	<code>[H: 你好..., A1: 你好老王...]</code> (2)	存储到 Memory 中，等待下次	-
2	<code>invoke("从现在起，你叫小王")</code>	<code>[H: 你好..., A1: ..., H: 从现在起...]</code> (3)	不触发 (长度 ≤ 3)	<code>[H: 你好..., A1: ..., H: 从现在起...]</code>
--	模型自动回复 (A2)	<code>[H: 你好..., A1: ..., H: 从现在..., A2: 好的...]</code> (4)	存储到 Memory 中，等待下次	-
3	<code>invoke("今天天气不错")</code>	<code>[H: 你好..., A1: ..., H: 从现在..., A2: ..., H: 今天天气...]</code> (5)	触发裁剪！长度 5 是奇数 $\rightarrow$ 取后 4 条 (<code>messages[-4:]</code>)	<code>[H: 你好，我是老王]</code> (第一条) + <code>[A1, H: 从现在..., A2, H: 今天天气...]</code> (后4条)
--	模型自动回复 (A3)	此时 State 被重置为上述 5 条 + 新回复的 A3 (共 6 条)	存储到 Memory 中	-
4	<code>invoke("告诉我，你是谁？我是谁？")</code>	<code>[H: 你好..., A1: ..., H: 从现在..., A2: ..., H: 今天..., A3: ..., H: 告诉我...]</code> (7)	触发裁剪！长度 7 是奇数 $\rightarrow$ 取后 4 条 (<code>messages[-4:]</code>)	<code>[H: 你好，我是老王]</code> (第一条) + <code>[A2, H: 今天..., A3, H: 告诉我...]</code> (后4条)

2.4.2 消息删除

消息裁剪强调“在 `模型调用前裁剪` 消息列表，**控制模型可以看到的上下文范围**”，而消息删除强调 `模型调用完成后将某些消息从消息列表中移除`，永久更改状态。

适合明确要遗忘、清理、重置某些历史。

```
1 from langchain.messages import RemoveMessage
2 from langchain.agents import create_agent, AgentState
3 from langchain.agents.middleware import after_model
4 from langgraph.checkpoint.memory import InMemorySaver
5 from langgraph.runtime import Runtime
6 from langchain_core.runnables import RunnableConfig
7
8
9 @after_model
10 def delete_old_messages(state: AgentState, runtime: Runtime) -> dict | None:
```

```

11     messages = state["messages"]
12     # 保持最近的 5 条消息
13     if len(messages) > 5:
14         # 框架中通常使用 RemoveMessage 来标记删除，并返回更新状态。
15         to_delete = len(messages) - 5
16         return {"messages": [RemoveMessage(id=m.id) for m in
messages[:to_delete]]}
17     return None
18
19
20 agent = create_agent(
21     model=model,
22     middleware=[delete_old_messages],
23     checkpointer=InMemorySaver()
24 )
25
26 config: RunnableConfig = {"configurable": {"thread_id": "1"}}
27
28 agent.invoke({"messages": "你好，我是老王"}, config)
29 agent.invoke({"messages": "从现在起，你叫小王"}, config)
30 agent.invoke({"messages": "今天天气不错"}, config)
31 final_response = agent.invoke({"messages": "告诉我，你是谁？我是谁？"}, config)
32
33 for msg in final_response["messages"]:
34     msg.pretty_print()

```

输出

```

1  ===== Ai Message
2  =====
3  好的，从现在起你可以叫我小王。
4  ===== Human Message
5  =====
6  今天天气不错
7  ===== Ai Message
8  =====
9  是啊，天气不错的时候心情也容易变好。你今天打算出去走走，还是在家休息一下？
10 ===== Human Message
11 =====
12 告诉我，你是谁？我是谁？
13 ===== Ai Message
14 =====
15 我是小王，你是老王。

```

显然，模型调用后消息列表规模超过阈值触发了消息裁剪。

说明：只要消息总数超过 5 条，就计算超出几条（`to_delete`），然后精准地删掉最老的那几条，使剩下的消息总数永远保持在 5 条。

分析消息列表 `messages` 的数量变化：

📁 第一轮：

- 用户说: "你好, 我是老王" (第 1 条)
- AI 回复: "你好, 老王!" (第 2 条)
- 数量: 此时 `len(messages) == 2` (不大于 5)。
- 结果: 中间件不触发。 (**记忆: 记住你是老王**)

📁 第二轮:

- 用户说: "从现在起, 你叫小王" (第 3 条)
- AI 回复: "好的, 我是小王。" (第 4 条)
- 数量: 此时 `len(messages) == 4` (不大于 5)。
- 结果: 中间件不触发。 (**记忆: 记住你是老王、我是小王**)

📁 第三轮:

- 用户说: "今天天气不错" (第 5 条)
- AI 回复: "是啊, 天气好心情也好。" (第 6 条)
- 数量: 此时 `len(messages) == 6`。触发中间件条件 (`> 5`)!
- 清理发生:
 - 计算 `to_delete = 6 - 5 = 1`。
 - 清理 `messages[:1]`, 也就是只删除了第 1 条消息 ("你好, 我是老王")。
- 剩余消息 (共 5 条):
 1. AI 回复: "你好, 老王!" (第 2 条)
 2. 用户说: "从现在起, 你叫小王" (第 3 条)
 3. AI 回复: "好的, 我是小王。" (第 4 条)
 4. 用户说: "今天天气不错" (第 5 条)
 5. AI 回复: "是啊, 天气好心情也好。" (第 6 条)

📁 第四轮 (最终提问):

- 用户输入: "告诉我, 你是谁? 我是谁?" (第 7 条)
 - 大模型读取上下文: 在 AI 做出最终回复前, 大模型会读取当前留存的 5 条历史记录 + 用户的这句新提问。
 - 大模型通过上下文里 AI 自己说过的"你好, 老王!", 依然能够推断出"哦, 原来坐在对面的人叫老王"。
- AI 做出最终回复: 比如 "你是老王, 我是小王啊。" (第 8 条)
- 数量: 此时回复完后, 总数 `len(messages) == 8`。再次触发中间件!
- 清理发生:
 - 计算 `to_delete = 8 - 5 = 3`。
 - 清理 `messages[:3]`, 即把目前列表中最老的前 3 条消息删除。
- 剩余消息: 依然精准保持在最后 5 条。
 - AI 回复: "好的, 我是小王。" (第 4 条)
 - 用户说: "今天天气不错" (第 5 条)
 - AI 回复: "是啊, 天气好心情也好。" (第 6 条)
 - 用户说: 告诉我, 你是谁? 我是谁? (第 7 条)
 - AI 回复: 你是老王, 我是小王啊。 (第 8 条)

RemoveMessage到底干了什么?

当你在中间件里返回 `[RemoveMessage(id=m.id)]` 时, 你实际上是向框架发送了一个 **删除指令**。

框架的底层处理逻辑如下:


```

1 [历史消息池（内存中持续存在）]
2   |— Message(id="1", content="你好，我是老王")
3   |— Message(id="2", content="...")
4   |— RemoveMessage(id="1") <-- 这是一个新追加进去的“墓碑”标记

```

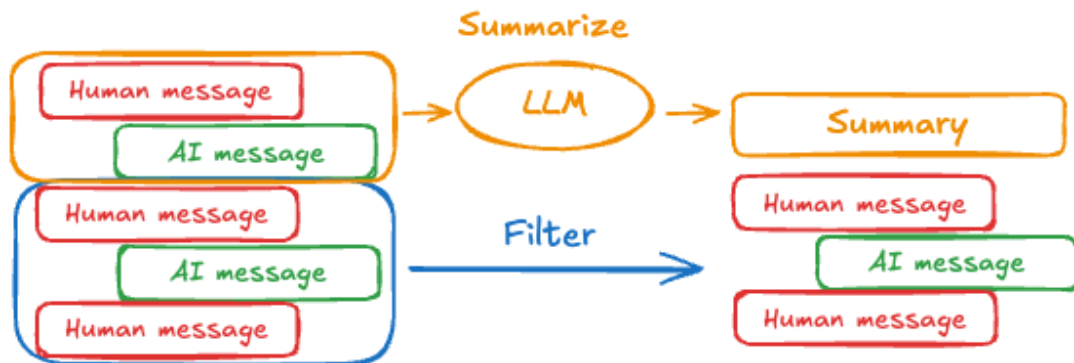
1. **追加“墓碑”标记**：框架收到 `RemoveMessage(id="1")` 后，并不会去内存的数组里把 `id="1"` 的对象删掉，而是把这个 `RemoveMessage` 作为一条新记录追加到当前线程的状态历史中。这个 `RemoveMessage` 就像是一个“墓碑”。
2. **运行时过滤合并 (Reducer)**：当下一次你再次调用 `agent.invoke` 或者大模型要去读取上下文时，框架的内置合并器 (Reducer) 会把“原始消息”和“墓碑标记”放在一起进行计算：

原始消息 (id: 1) + 墓碑标记 (id: 1) = $\emptyset$ (对外隐藏)

它在丢给大模型之前，会自动把被标记删除的消息过滤掉。

2.4.3 摘要

把早期历史压缩成摘要，再替换原始消息。



消息裁剪和删除都会导致上下文缺失，影响回答质量和用户体验。和它们相比，摘要更适合长会话的折中方案：保语义，不保原文。官方推荐内置 `SummarizationMiddleware`。上一章已有讲解。

```

1 from langchain.chat_models import init_chat_model
2 from dotenv import load_dotenv
3 import os
4
5 # 从.env文件中加载环境变量
6 load_dotenv(override=True)
7
8 model_out = init_chat_model(
9     model="gpt-5.4-mini",
10    model_provider="openai",
11    api_key=os.getenv("CLOSEAI_API_KEY"),
12    base_url=os.getenv("CLOSEAI_BASE_URL")
13 )

```

```

1 from langchain.chat_models import init_chat_model
2 from dotenv import load_dotenv
3 import os
4
5 # 从.env文件中加载环境变量
6 load_dotenv(override=True)
7
8 model_in = init_chat_model(
9     model="gpt-4o-mini",
10    model_provider="openai",
11    api_key=os.getenv("CLOSEAI_API_KEY"),
12    base_url=os.getenv("CLOSEAI_BASE_URL")
13 )

```

```

1 from langchain.agents.middleware import SummarizationMiddleware
2 from langchain.agents import create_agent
3 from langgraph.checkpoint.memory import InMemorySaver
4
5
6 # 创建带摘要中间件的 Agent
7 agent = create_agent(
8     model=model_out,
9     tools=[],
10    checkpointer=InMemorySaver(),
11    middleware=[
12        SummarizationMiddleware(
13            model=model_in,
14            trigger=[
15                ("tokens", 100), # 超过 100 tokens 就摘要
16            ],
17            keep=("messages", 2),
18            summary_prompt="对历史消息摘要，消息列表如下\n{messages}",
19        )
20    ]
21 )
22
23
24 config = {"configurable": {"thread_id": "1"}}
25
26 print("\n进行多轮对话...")
27 conversations = [
28     "我叫张三，是工程师。这里是一段非常长非常长的废话..." * 20, # 强制撑爆 100 tokens
29     "请总结一下我的信息"
30 ]
31
32 for msg in conversations:
33     response = agent.invoke(
34         {"messages": [{"role": "user", "content": msg}]},
35         config=config
36     )
37     for msg in response["messages"]:
38         msg.pretty_print()
39     print("*" * 50)

```

```
2 ===== Human Message
3 =====
4 我叫张三，是工程师。这里是一段非常长非常长的废话...我叫张三，是工程师。这里是一段非常
  长非常长的废话...我叫张三，是工程师。这里是一段非常长非常长的废话...我叫张三，是工程
  师。这里是一段非常长非常长的废话...我叫张三，是工程师。这里是一段非常长非常长的废
  话...我叫张三，是工程师。这里是一段非常长非常长的废话...我叫张三，是工程师。这里是一
  段非常长非常长的废话...我叫张三，是工程师。这里是一段非常长非常长的废话...我叫张三，
  是工程师。这里是一段非常长非常长的废话...我叫张三，是工程师。这里是一段非常长非常长的
  废话...我叫张三，是工程师。这里是一段非常长非常长的废话...我叫张三，是工程师。这里
  是一段非常长非常长的废话...我叫张三，是工程师。这里是一段非常长非常长的废话...我叫张
  三，是工程师。这里是一段非常长非常长的废话...我叫张三，是工程师。这里是一段非常长非常
  长的废话...我叫张三，是工程师。这里是一段非常长非常长的废话...我叫张三，是工程师。这
  里是一段非常长非常长的废话...我叫张三，是工程师。这里是一段非常长非常长的废话...我叫
  张三，是工程师。这里是一段非常长非常长的废话...我叫张三，是工程师。这里是一段非常长非
  常长的废话...
5 ===== Ai Message
6 =====
7 你好，张三。你想让我基于这段内容做什么？
8 如果你愿意，我可以帮你：
9
10 1. **提炼关键信息**
11 2. **总结成一句话/一段话**
12 3. **改写成更正式自我介绍**
13 4. **识别并去掉重复废话**
14 5. **翻译成英文**
15
16 例如，提炼后可以写成：
17
18 > 张三是一名工程师。
19
20 如果你告诉我目标，我可以直接帮你处理。
21 *****
22 ===== Human Message
23 =====
24 Here is a summary of the conversation to date:
25
26 根据历史消息，用户“张三”自称是一名工程师，并重复发送了大量无实质内容的废话（如“这里
  是一段非常长非常长的废话”）。摘要如下：
27
28 **摘要**：用户张三（工程师）发送了重复的冗长废话，内容无实际信息。
29 ===== Ai Message
30 =====
31 你好，张三。你想让我基于这段内容做什么？
32 如果你愿意，我可以帮你：
33
34 1. **提炼关键信息**
35 2. **总结成一句话/一段话**
36 3. **改写成更正式自我介绍**
37 4. **识别并去掉重复废话**
38 5. **翻译成英文**
39
40 例如，提炼后可以写成：
41
42 > 张三是一名工程师。
```

```

43
44 如果你告诉我目标，我可以直接帮你处理。
45 ===== Human Message
46 =====
47 请总结一下我的信息
48 ===== Ai Message
49 =====
50 根据你提供的内容，目前能总结出的信息是：
51
52 - **姓名**：张三
53 - **职业**：工程师
54 - **特点**：之前发送的内容以重复、冗长的无实质信息为主
55
56 如果你愿意，我还可以帮你把这些信息整理成：
57 - **正式简介**
58 - **一句话自我介绍**
59 - **简历风格描述**
60 - **更自然的个人介绍**
61 *****

```

```

1 final_state = agent.get_state(config)
2 print("\n--- 最终保存在 State 中的真实消息 ---")
3 for msg in final_state.values.get("messages", []):
4     msg.pretty_print()

```

```

1 --- 最终保存在 State 中的真实消息 ---
2 ===== Human Message
3 =====
4 Here is a summary of the conversation to date:
5
6 根据历史消息，用户“张三”自称是一名工程师，并重复发送了大量无实质内容的废话（如“这里是一段非常长非常长的废话”）。摘要如下：
7
8 **摘要**：用户张三（工程师）发送了重复的冗长废话，内容无实际信息。
9 ===== Ai Message
10 =====
11 你好，张三。你想让我基于这段内容做什么？
12 如果你愿意，我可以帮你：
13
14 1. **提炼关键信息**
15 2. **总结成一句话/一段话**
16 3. **改写成更正式自我介绍**
17 4. **识别并去掉重复废话**
18 5. **翻译成英文**
19
20 例如，提炼后可以写成：
21
22 > 张三是一名工程师。
23
24 如果你告诉我目标，我可以直接帮你处理。
25 ===== Human Message
26 =====

```

```
26
27 请总结一下我的信息
28 ===== Ai Message
29 =====
30 根据你提供的内容，目前能总结出的信息是：
31
32 - **姓名**：张三
33 - **职业**：工程师
34 - **特点**：之前发送的内容以重复、冗长的无实质信息为主
35
36 如果你愿意，我还可以帮你把这些信息整理成：
37 - **正式简介**
38 - **一句话自我介绍**
39 - **简历风格描述**
40 - **更自然的个人介绍**
```

工作原理

```
1 对话历史：[消息1，消息2，...，消息20] （超过 100 tokens）
2   ↓
3 SummarizationMiddleware 自动触发
4   ↓
5 摘要旧消息："用户是张三，在北京工作，喜欢编程..."
6   ↓
7 新历史：[摘要，最近消息] （减少到 100 tokens）
```

常见问题

1. 摘要会丢失信息吗？

会有一些细节丢失，但：

- 重要信息会保留（姓名、关键事实）
- 最近的消息完整保留
- 对于大部分场景足够

2. 设置最大token数触发摘要的标准是啥？

```
1 # 模型上下文窗口 4k → 设置 3000
2 # 模型上下文窗口 8k → 设置 6000
3 # 模型上下文窗口 16k → 设置 12000
4
5 # 留一些余量给工具调用和系统提示
```

3. 摘要成本高吗？

- 摘要只在超过阈值时触发
- 可以使用便宜的模型（如 gpt-4o-mini）
- 相比传输全部历史，通常更便宜

4. 摘要触发的频率要关注吗？

要关注，根据监控摘要触发频率，调整阈值。

- 如果频繁触发 → 提高阈值
- 如果从不触发 → 降低阈值

2.4.4 自定义过滤策略

通过 `中间件` 可以随意更改消息列表，因此可以实现任意的过滤策略。

此处不再演示。

2.5 了解：state的理解

state是agent底层有状态运行图的状态信息，是AgentState类型的实例，定义如下：

```
1 class AgentState(TypedDict, Generic[ResponseT]):
2     """State schema for the agent."""
3
4     messages: Required[Annotated[list[AnyMessage], add_messages]]
5     jump_to: NotRequired[Annotated[JumpTo | None, EphemeralValue,
6     PrivateStateAttr]]
7     structured_response: NotRequired[Annotated[ResponseT, OmitFromInput]]
```

说明：

- AgentState是TypedDict的子类，这意味着我们可以按照 `字典的读写方式` 访问其实例的元素。
- 该类有三个字段
 - `messages`：截止到当前节点的历史会话消息记录，标记为Required。
 - `jump_to`：中间件章节介绍过，表示跳转至运行图的指定节点，标记为NotRequired，表示该字段可以为None。（后续案例可以看到这一点）
 - `structured_response`：结构化输出内容，当我们启用结构化输出时，结构化后的内容会被记录在这里，见下文案例。

举例：

```
1 from langchain.chat_models import init_chat_model
2 from dotenv import load_dotenv
3 import os
4
5 # 从.env文件中加载环境变量
6 load_dotenv(override=True)
7
8 model = init_chat_model(
9     model="gpt-5.4-mini",
10    model_provider="openai",
11    api_key=os.getenv("CLOSEAI_API_KEY"),
12    base_url=os.getenv("CLOSEAI_BASE_URL")
13 )
```

```
1 from langchain.agents import create_agent
2 from langchain.agents.middleware import AgentState, before_model,
3 wrap_tool_call, after_agent
4 from langchain.tools.tool_node import ToolCallRequest
5 from langchain.messages import HumanMessage, AIMessage, ToolMessage
6 from langchain.tools import tool
7 from langgraph.runtime import Runtime
8 from langgraph.types import Command
9 from typing import Any, Callable
10 from pydantic import BaseModel, Field
```

```

11
12 class WeatherInfo(BaseModel):
13     """城市天气情况"""
14     city: str = Field(description="城市名称")
15     temperature: str = Field(description="气温")
16     desc: str = Field(description="当日天气概述")
17
18 @tool(parse_docstring=True)
19 def get_weather(city: str):
20     """
21     获取当日天气
22
23     Args:
24         city: 城市名称
25     """
26     return f"[{city}] 今天气温9~16度，万里无云，天气不错适合外出"
27
28 @before_model(can_jump_to=["tools"])
29 def direct_tool_call(state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
30     last_msg = state["messages"][-1]
31     if isinstance(last_msg, HumanMessage) and "天气" in last_msg.text and "北
32     京" in last_msg.text:
33         fake_tool_call = AIMessage(
34             content="人工构造的消息",
35             tool_calls=[
36                 {
37                     "name": "get_weather",
38                     "args": {
39                         "city": "北京"
40                     },
41                     "id": "direct_call_id"
42                 }
43             ]
44         )
45         return {
46             "messages": [fake_tool_call],
47             "jump_to": "tools"
48         }
49     return None
50
51 @wrap_tool_call
52 def first_check(
53     request: ToolCallRequest,
54     handler: Callable[[ToolCallRequest], ToolMessage | Command]
55 ) -> ToolMessage | Command:
56     print("=" * 30, '-> In first_check Middleware <-', "=" * 30)
57     print(f"{request.state.get('jump_to', None) = }")
58     print(f"{request.state.get('structured_response', None) = }")
59
60     return handler(request)
61
62 @after_agent
63 def final_check(state: AgentState, runtime: Runtime) -> dict[str, Any] |
64     None:
65     print("=" * 30, '-> In final_check Middleware <-', "=" * 30)
66     for msg in state["messages"]:

```

```

66         msg.pretty_print()
67
68         print("=" * 30, "-> 消息打印完毕 <=", "=" * 30)
69         print(f"{state.get('jump_to', None) = }")
70         print(f"{state.get('structured_response', None) = }")
71
72         return None
73
74     agent = create_agent(
75         model = model,
76         response_format=weatherInfo,
77         middleware=[direct_tool_call, first_check, final_check],
78         tools=[get_weather],
79     )
80
81     response = agent.invoke({
82         "messages": [
83             HumanMessage("请帮我查询北京当日天气")
84         ]
85     })

```

输出

```

1  ===== -> In first_check Middleware <-
2  =====
3  request.state.get("jump_to", None) = 'tools'
4  request.state.get("structured_response", None) = None
5  ===== -> In final_check Middleware <-
6  =====
7  ===== Human Message
8  =====
9
10 请帮我查询北京当日天气
11 ===== Ai Message
12 =====
13
14 人工构造的消息
15 Tool Calls:
16 get_weather (direct_call_id)
17 Call ID: direct_call_id
18 Args:
19   city: 北京
20 ===== Tool Message
21 =====
22 Name: get_weather
23
24 [北京] 今天气温9~16度, 万里无云, 天气不错适合外出
25 ===== Ai Message
26 =====
27 {"city": "北京", "temperature": "9~16℃", "desc": "万里无云, 天气不错适合外出"}
28 ===== -> 消息打印完毕 <-
29 =====
30 state.get("jump_to", None) = None
31 state.get("structured_response", None) = weatherInfo(city='北京',
32 temperature='9~16℃', desc='万里无云, 天气不错适合外出')

```


分析

1. 我们在模型调用之前通过jump_to直接跳转至工具节点
2. 在工具调用前打印状态信息，此时的jump_to非空
3. 在Agent执行完毕之后打印状态信息，此时的structure_response非空而jump_to为空。

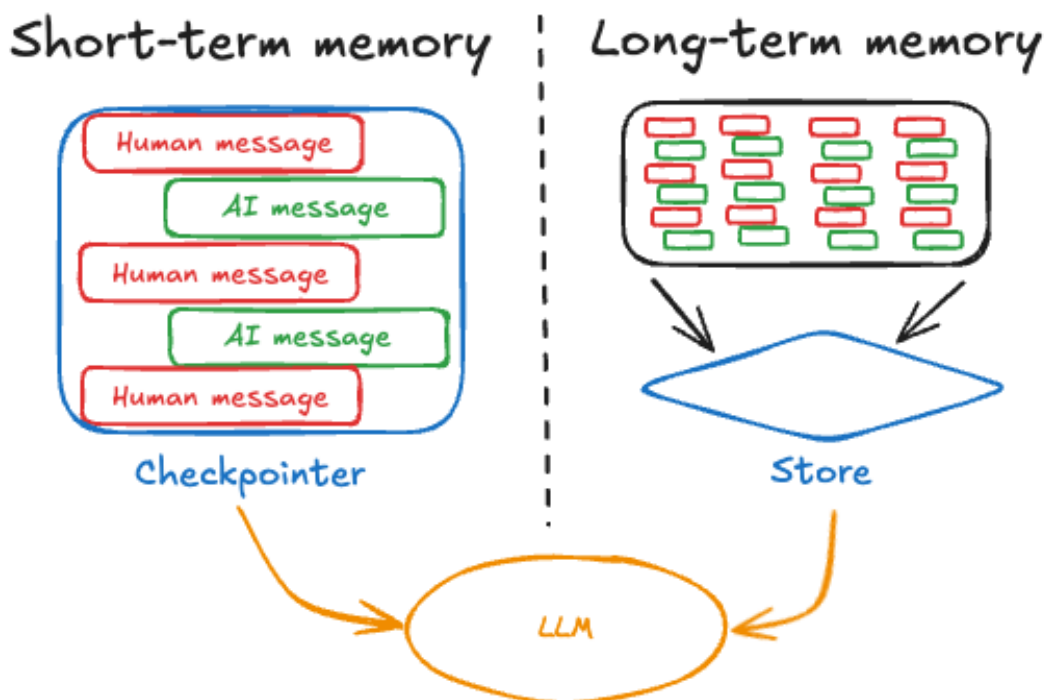
3、长期记忆

3.1 基本理解

3.1.1 什么是长期记忆

短期记忆记录的是 **会话级别（线程，Thread）** 的数据，**会话间不共享**。

而长期记忆记录的是用户特定或应用级别的数据，任何会话都可以随时访问。



比如：

- 你喜欢简短回答
- 你偏好 Python
- 某个用户是 VIP
- 某个流程过去怎么做效果更好

这类信息不属于某一条聊天，而属于“用户/组织/应用本身”。

3.1.2 类型划分

LangChain参考[CoALA paper](#)将长期记忆划分为三类：

Memory Type	存什么
Semantic （语义记忆）	事实
Episodic （情景记忆）	经验
Procedural （程序性记忆）	规则/做事方法

类型1: Semantic Memory (语义记忆)

即“事实类记忆”，记录事实/用户偏好/概念，如：

- 用户喜欢简洁回答
- 用户常用中文
- 某个公司属于哪个行业

类型2: Episodic Memory (情景记忆)

即“经验类记忆”，记录Agent过去执行的动作，如：

- 过去某个任务是怎么成功的
- 某种用户输入下，怎样回答效果最好

在Agent里，这常常表现为 **few-shot examples (少样本示例)**：

不直接告诉模型规则，而是给它看几个“输入 -> 输出”的例子，让它照着学

类型3: Procedural Memory (程序性记忆)

即“规则/做事方法”，如

- Agent的系统提示词
- Agent的工作流程
- 工具调用规则

3.1.3 存储架构

长期记忆的存储是 **store -> namespace -> key -> value** 的四层架构。

第1层: Store (记忆仓库)

- Store是 `langgraph.store.base.BaseStore` 的子类实例，由全类名可知，store是由LangGraph提供的。

- 常用实现类：

`InMemoryStore`：将长期记忆存储在内存，适合测试

`PostgresStore`：将长期记忆存储在外部的PostgreSQL数据库，适合生产环境

- 开发期可用 `InMemoryStore`；生产建议数据库后端，如 `PostgresStore`

第2层: Namespace (命名空间)

数据类型是由任意长度的 `tuple[str, ...]` 表示的 **层级路径**。作用上很像“文件路径 / 文件夹层级”，用于给长期记忆分组和隔离。数据类型为 **字符串元组**

第3层: Key (键)

是该 namespace 下的唯一标识，单条记忆的唯一键，数据类型为 **字符串(str)**

第4层: Value (值)

是存储的值，数据类型为 **字典(dict[str, Any])**

举例1:

每个namespace存储的都是key-value键值对，通过key可以唯一标识一条value。

```

1 namespace = ("users", "user_123", "preferences") # 元组类型
2 key = "profile" # 字符串类型
3 value = { # 字典类型
4     "language": "zh-CN",
5     "style": "short_direct",
6     "likes": ["python", "rag"]
7 }
8 store.put(namespace, key, value)

```

举例2:

同一个 AI 应用通常会为每个独立会话维护各自的短期状态 **State**；而长期记忆通常可以共享同一个 **Store** 实例，再通过 **namespace** 区分不同用户、组织、业务域或会话相关数据。如下：

```

1 AI应用
2 └─ thread_id = t1 -> state_1
3   │ └─ messages = [
4   │ │   {"role": "user", "content": "我想去北京旅游"},
5   │ │   {"role": "assistant", "content": "你想玩几天? "}
6   │ │ ]
7   │ └─ current_intent = "travel_planning"
8   │   └─ collected_slots = {"destination": "北京"}
9   │
10  └─ thread_id = t2 -> state_2
11    │ └─ messages = [
12    │ │   {"role": "user", "content": "帮我写周报"}
13    │ │ ]
14    │ └─ current_intent = "write_report"
15    │   └─ collected_slots = {}
16    │
17  └─ thread_id = t3 -> state_3
18    │ └─ messages = [
19    │ │   {"role": "user", "content": "我喜欢简洁风格的UI"}
20    │ │ ]
21    │ └─ current_intent = "ui_design"
22    │   └─ collected_slots = {"style": "minimal"}
23    │
24    └─ shared store
25        │ └─ namespace = (user_1, "memories")
26        │   │ └─ key = "profile"
27        │   │   │ └─ value = {
28        │   │   │ │   "name": "张三",
29        │   │   │ │   "city": "上海",
30        │   │   │ │   "preferences": ["简洁风格", "中文回复"]
31        │   │   │ }
32        │   │ └─ key = "travel_preference"
33        │   │   │ └─ value = {
34        │   │   │ │   "favorite_cities": ["北京", "杭州"],
35        │   │   │ │   "budget_level": "medium"
36        │   │   │ }
37        │   │   └─ key = "writing_style"
38        │   │     │ └─ value = {
39        │   │     │ │   "tone": "professional",
40        │   │     │ │   "language": "zh-CN"
41        │   │     │ }
42        │   │
43        └─ namespace = (user_2, "memories")

```

```

44 | | | key = "profile"
45 | | | value = {
46 | | |     "name": "李四",
47 | | |     "city": "深圳"
48 | | | }
49 | | | key = "product_interest"
50 | | | value = {
51 | | |     "topics": ["AI Agent", "RAG", "workflow"]
52 | | | }
53 | | |
54 | | | namespace = (user_1, thread_3, "artifacts")
55 | | | | key = "draft_v1"
56 | | | | value = {
57 | | | |     "type": "html",
58 | | | |     "content": "<html>...</html>"
59 | | | | }
60 | | | | key = "ui_notes"
61 | | | | value = {
62 | | | |     "summary": "用户偏好留白多、低饱和配色"
63 | | | | }
64 | | | | key = "final_scheme"
65 | | | | value = {
66 | | | |     "palette": ["#F5F1E8", "#1F3A5F", "#D97B2D"],
67 | | | |     "font_style": "clean"
68 | | | | }

```

3.2 基础API的使用

LangChain 1.2.x 的长期记忆基于 **store** 持久化数据，相关的API有：

- **put()**：负责写入
- **get()**：负责读取
- **search()**：负责检索

我们可以在Agent执行**流程之外**直接访问长期记忆。

3.2.1 put()/get()：写入/读取API

① put()源码剖析

直接调用**put()**函数即可，函数签名如下

```

1 def put(
2     self,
3     namespace: tuple[str, ...],
4     key: str,
5     value: dict[str, Any],
6     index: Literal[False] | list[str] | None = None,
7     *,
8     ttl: float | None | NotProvided = NOT_PROVIDED,
9 ) -> None:

```

参数说明

- **namespace**: 文档所在的层级路径

- **key**: 该路径下的唯一键
- **value**: 要保存的 JSON-like 字典
- **index**: 控制语义检索索引
 - **None(默认选项)**: 使用 store 初始化时配置的索引配置，如果初始化时没有指定索引策略，则 **index** 参数将会被忽略
 - **False**: 不为该 item 建立语义索引
 - **list[str]**: 只对指定字段路径建索引
- **ttl**: 可选，过期时间；是否支持取决于具体 store 实现

举例：

```
1 store.put(
2     ("users", "alice", "memories"), # namespace
3     "pref_food",                    # key
4     {"category": "food", "text": "Alice likes sushi"} # value
5 )
```

② get()源码剖析

按照 **namespace + key** 精确查询，返回的不止是 **value**，而是完整对象。即LangGraph底层将数据封装为 **Item 对象**。

函数签名如下

```
1 def get(
2     self,
3     namespace: tuple[str, ...],
4     key: str,
5     *,
6     refresh_ttl: bool | None = None,
7 ) -> Item | None:
```

参数说明

- **namespace**: 文档所在的层级路径
- **key**: 该路径下的唯一键
- **refresh_ttl**: 是否刷新当前item的ttl (**time-to-live, 存活时间**)
 - 默认为**None**：表示采用创建store对象时指定的同名配置
 - 如果没有配置TTL，该参数被忽略。

举例：

```
1 item = my_store.get(("users", "alice", "memories"), "pref_food")
2
3 if item is not None:
4     print(item.value)
5     # {'category': 'food', 'text': 'Alice likes sushi'}
```

③ 举例1: 基于InMemoryStore

举例1: 添加

```
1 from langgraph.store.memory import InMemoryStore
2
3 store = InMemoryStore()
4
5 namespace = ("users",)
6 user_id = 'user-1'
7 username = "小蓝"
8 store.put(namespace, user_id, {"name": username})
9
10 print(store.get(namespace, user_id))
```

输出

```
1 Item(namespace=['users'], key='user-1', value={'name': '小蓝'},
  created_at='2026-06-11T15:20:52.542493+00:00', updated_at='2026-06-11T15:20:52.542494+00:00')
```

注意到, Item对象新增了 `created_at` 和 `updated_at` 字段, 分别为数据新增和更改时间。

注意: 对于当前版本, InMemoryStore每次put都会创建一个新的Item对象, 无论namespace和key是否相同, 所以 `created_at` 和 `updated_at` 始终是一致的。

举例2: 更新

对同一条数据进行更改后查询。

```
1 store.put(namespace, user_id, {"name": '小红'})
2 print(store.get(namespace, user_id))
```

输出

```
1 Item(namespace=['users'], key='user-1', value={'name': '小红'},
  created_at='2026-06-11T15:23:11.283981+00:00', updated_at='2026-06-11T15:23:11.283982+00:00')
```

④ 举例2: 基于PostgresStore

PostgresStore不同于InMemoryStore, 更改数据的逻辑是update而非覆盖, 因此 `created_at` 固定为Item创建时间, 而 `updated_at` 为更新时间, 二者可能不同, 符合直觉。

举例1: 添加

```

1 from langgraph.store.postgres import PostgresStore
2
3 namespace = ("users", )
4 user_id = "user-11"
5 username = "小蓝"
6
7 DB_URL =
8     "postgresql://langchain_user:abcd1234@118.195.128.47:5432/langchain_db?
9     sslmode=disable"
10
11 with PostgresStore.from_conn_string(DB_URL) as store:
12     store.setup()
13     store.put(namespace, user_id, {"name": username})
14     print(store.get(namespace, user_id))

```

```

1 Item(namespace=['users'], key='user-11', value={'name': '小蓝'},
2     created_at='2026-06-11T23:24:31.132342+08:00', updated_at='2026-06-
3     11T23:24:31.132342+08:00')

```

说明：这里的DB_URL大家测试时，需要替换为自己的。

举例2：更新

更改同一条数据

```

1 with PostgresStore.from_conn_string(DB_URI) as store:
2     store.setup()
3     store.put(namespace, user_id, {"name": "小红"})
4     print(store.get(namespace, user_id))

```

输出

```

1 Item(namespace=['users'], key='user-11', value={'name': '小红'},
2     created_at='2026-06-11T23:24:31.132342+08:00', updated_at='2026-06-
3     11T23:26:11.956257+08:00')

```

可以看到，`created_at` 没变，但 `updated_at` 更改了。

3.2.3 search(): 检索API

① 源码剖析

函数签名如下

```

1 def search(
2     self,
3     namespace_prefix: tuple[str, ...],
4     /,
5     *,
6     query: str | None = None,
7     filter: dict[str, Any] | None = None,
8     limit: int = 10,
9     offset: int = 0,
10    refresh_ttl: bool | None = None,
11 ) -> list[SearchItem]:

```

参数说明：

- **namespace_prefix**: 命名空间前缀, 在该前缀下搜索。
- **query**: 语义检索时用于查询的自然语言。
- **filter**: 过滤条件, value中的键值对组合, 见下文举例。
- **limit**: 可以返回item的**最大条数**, 效果等同于SQL中的**limit**。
- **offset**: 返回结果之前跳过的item数量。
- **refresh_ttl**: 同上。

它支持两种检索方式 (对应上面的参数2、3) :

- 按 **filter 做结构化过滤**, 即用 value 中的 **键值** 筛选符合条件的记录。
- 按 **query 做语义相似度检索**, 需要将输入转换为向量。

返回值:

返回匹配的 **SearchItem** 列表, 并额外携带匹配分数等检索元信息。

② 举例1: 按照namespace前缀搜索

本节全部基于 **InMemoryStore** 测试。

1、准备store

```

1  from langgraph.store.memory import InMemoryStore
2
3  store = InMemoryStore()
4
5  namespace1 = ("users", "Alice", "memories")
6  key1 = 'preferences'
7  value1 = {
8      "course": "计算机组成原理",
9      "sports": "跑步",
10     "food": "紫光园奶皮子酸奶"
11 }
12
13 namespace2 = ("users", "Bob", "memories")
14 key2 = 'preferences'
15 value2 = {
16     "course": "数字电路与模拟电路",
17     "sports": "跑步",
18     "food": "奶皮子糖葫芦"
19 }
20
21 namespace3 = ("users", "Black", "memories")
22 key3 = 'preferences'
23 value3 = {
24     "course": "数字电路与模拟电路",
25     "sports": "羽毛球",
26     "food": "紫光园奶皮子酸奶"
27 }
28
29 store.put(namespace1, key1, value1)
30 store.put(namespace2, key2, value2)
31 store.put(namespace3, key3, value3)

```

如果在jupyter环境, 执行上述代码后, 直接执行后续代码即可。

如果每次测试都在单独的python文件中进行, 则以下1、2小节的案例代码之前都要补充上述代码。

2、按照namespace前缀搜索

代码1

```
1 print('=' * 30, '-> (users, ) <-', "=" * 30)
2 for item in store.search(("users", )):
3     print(item)
```

输出

```
1 ===== -> (users, ) <-
2 =====
3 Item(namespace=['users', 'Alice', 'memories'], key='preferences', value=
4 {'course': '计算机组成原理', 'sports': '跑步', 'food': '紫光园奶皮子酸奶'},
   created_at='2026-06-11T15:27:24.734788+00:00', updated_at='2026-06-
   11T15:27:24.734789+00:00', score=None)
5 Item(namespace=['users', 'Bob', 'memories'], key='preferences', value=
6 {'course': '数字电路与模拟电路', 'sports': '跑步', 'food': '奶皮子糖葫芦'},
   created_at='2026-06-11T15:27:24.734809+00:00', updated_at='2026-06-
   11T15:27:24.734810+00:00', score=None)
7 Item(namespace=['users', 'Black', 'memories'], key='preferences', value=
8 {'course': '数字电路与模拟电路', 'sports': '羽毛球', 'food': '紫光园奶皮子酸
   奶'}, created_at='2026-06-11T15:27:24.734829+00:00', updated_at='2026-06-
   11T15:27:24.734829+00:00', score=None)
```

代码2

```
1 print('=' * 30, '-> (users, Alice) <-', "=" * 30)
2 for item in store.search(("users", "Alice")):
3     print(item)
```

输出

```
1 ===== -> (users, Alice) <-
2 =====
3 Item(namespace=['users', 'Alice', 'memories'], key='preferences', value=
4 {'course': '计算机组成原理', 'sports': '跑步', 'food': '紫光园奶皮子酸奶'},
   created_at='2026-06-11T15:27:24.734788+00:00', updated_at='2026-06-
   11T15:27:24.734789+00:00', score=None)
```

③ 举例2: 按照filter过滤

代码1

```
1 print("=" * 30, '-> (users, ), filter=food <-', "=" * 30)
2 for item in store.search(("users", ), filter={"food": "紫光园奶皮子酸奶"}):
3     print(item)
```

输出

```

1  ===== -> (users, ), filter=food <-, "=" * 30)
2  Item(namespace=['users', 'Alice', 'memories'], key='preferences', value=
  {'course': '计算机组成原理', 'sports': '跑步', 'food': '紫光园奶皮子酸奶'},
  created_at='2026-06-11T15:27:24.734788+00:00', updated_at='2026-06-
  11T15:27:24.734789+00:00', score=None)
3  Item(namespace=['users', 'Black', 'memories'], key='preferences', value=
  {'course': '数字电路与模拟电路', 'sports': '羽毛球', 'food': '紫光园奶皮子酸
  奶'}, created_at='2026-06-11T15:27:24.734829+00:00', updated_at='2026-06-
  11T15:27:24.734829+00:00', score=None)

```

代码2

```

1  print("=" * 30, '-> (users, ), filter=sports <-, "=" * 30)')
2  for item in store.search(("users", ), filter={"sports": "跑步"}):
3      print(item)

```

输出

```

1  ===== -> (users, ), filter=sports <-, "=" *
  30)
2  Item(namespace=['users', 'Alice', 'memories'], key='preferences', value=
  {'course': '计算机组成原理', 'sports': '跑步', 'food': '紫光园奶皮子酸奶'},
  created_at='2026-06-11T15:27:24.734788+00:00', updated_at='2026-06-
  11T15:27:24.734789+00:00', score=None)
3  Item(namespace=['users', 'Bob', 'memories'], key='preferences', value=
  {'course': '数字电路与模拟电路', 'sports': '跑步', 'food': '奶皮子糖葫芦'},
  created_at='2026-06-11T15:27:24.734809+00:00', updated_at='2026-06-
  11T15:27:24.734810+00:00', score=None)

```

代码3

```

1  print("=" * 30, '-> (users, ), filter=course <-, "=" * 30)')
2  for item in store.search(("users", ), filter={"course": "数字电路与模拟电路"}):
3      print(item)

```

输出

```

1  ===== -> (users, ), filter=course <-, "=" *
  30)
2  Item(namespace=['users', 'Bob', 'memories'], key='preferences', value=
  {'course': '数字电路与模拟电路', 'sports': '跑步', 'food': '奶皮子糖葫芦'},
  created_at='2026-06-11T15:27:24.734809+00:00', updated_at='2026-06-
  11T15:27:24.734810+00:00', score=None)
3  Item(namespace=['users', 'Black', 'memories'], key='preferences', value=
  {'course': '数字电路与模拟电路', 'sports': '羽毛球', 'food': '紫光园奶皮子酸
  奶'}, created_at='2026-06-11T15:27:24.734829+00:00', updated_at='2026-06-
  11T15:27:24.734829+00:00', score=None)

```

④ 举例3: 按照语义搜索

举例1: 自定义嵌入函数

自定义嵌入函数，目的是查看嵌入向量

```

1  from langgraph.store.memory import InMemoryStore
2

```

```

3  # 自定义嵌入函数
4  def embed(text: list[str]) -> list[list[float]]:
5      return [[1.0] * 6 for _ in range(len(text))]
6
7  index_config = {
8      "embed": embed,
9      "dims": 6,
10     "fields": ["$", "course"]
11 }
12 store = InMemoryStore(
13     index = index_config
14 )
15
16 namespace1 = ("users", "Alice", "memories")
17 key1 = 'preferences'
18 value1 = {
19     "course": "计算机组成原理",
20     "sports": "跑步",
21     "food": "紫光园奶皮子酸奶"
22 }
23
24 namespace2 = ("users", "Bob", "memories")
25 key2 = 'preferences'
26 value2 = {
27     "course": "数字电路与模拟电路",
28     "sports": "跑步",
29     "food": "奶皮子糖葫芦"
30 }
31
32 namespace3 = ("users", "Black", "memories")
33 key3 = 'preferences'
34 value3 = {
35     "course": "数字电路与模拟电路",
36     "sports": "羽毛球",
37     "food": "紫光园奶皮子酸奶"
38 }
39
40 store.put(namespace1, key1, value1)
41 store.put(namespace2, key2, value2)
42 store.put(namespace3, key3, value3)

```

初始化Store时通过index指定索引方式，后者源码如下

```

1  class IndexConfig(TypedDict, total=False):
2      dims: int
3      embed: Embeddings | EmbeddingsFunc | AEmbeddingsFunc | str
4      fields: list[str] | None

```

- `embeds`：将输入文本转换为向量的嵌入函数，可以是 `自定义函数`，也可以是 `嵌入模型对象`，本例传递的是自定义嵌入函数。
- `dims`：输出向量维度
- `fields`：用于计算向量的属性列表，这里的属性都是指value中的key，value是一个JSON-like字典，可取值如下
 - `["$"]`：将value作为整体嵌入
 - `["fields1", "fields2"]`：单独指定某个一级字段

- `["parent.child"]` : 从内部的嵌套JSON对象中获取子字段的值
- `["array[*].field"]` : 从JSON数组的每个JSON对象中获取子字段的值
- **注意**: 上述四种形式可以同时出现, fields列表的**每个元素**都会生成**一个嵌入向量**。

查看嵌入向量

```
1 from pprint import pprint
2 pprint(store._vectors)
```

输出

```
1 defaultdict(<function InMemoryStore.__init__.<locals>.<lambda> at
0x000001c201898A40>,
2     {'users', 'Alice', 'memories': defaultdict(<class 'dict'>,
3     {'preferences':
4     {'$': [1.0,
5     1.0,
6     1.0,
7     1.0,
8     1.0,
9     1.0],
10    'course': [1.0,
11    1.0,
12    1.0,
13    1.0,
14    1.0]}}}),
15    ('users', 'Black', 'memories': defaultdict(<class 'dict'>,
16    {'preferences':
17    {'$': [1.0,
18    1.0,
19    1.0,
20    1.0,
21    1.0],
22    'course': [1.0,
23    1.0,
24    1.0,
```

```

25         1.0,
26
27         1.0,
28         1.0]]]),
29         ('users', 'Bob', 'memories'): defaultdict(<class 'dict'>,
30                                                     {'preferences':
31                                                         {'$': [1.0,
32                                                         1.0,
33                                                         1.0,
34                                                         1.0,
35                                                         1.0,
36                                                         1.0],
37                                                         'course': [1.0,
38                                                         1.0,
39                                                         1.0,
40                                                         1.0,
41                                                         1.0]]}}))

```

还可以通过指定namespace、key、index_config的fields中指定的字段名查看特定向量。

```

1 from pprint import pprint
2 pprint(store._vectors[('users', 'Alice', 'memories')]['preferences']['$'])

```

输出

```

1 [1.0, 1.0, 1.0, 1.0, 1.0, 1.0]

```

举例2：使用嵌入模型

本例要通过CloseAI平台调用OpenAI的嵌入模型 `openai:text-embedding-3-large`

其嵌入维度为3072，所以此处的 `dims` 应设置为3072

```

1 from langgraph.store.memory import InMemoryStore
2 from dotenv import load_dotenv
3 from langchain.embeddings import init_embeddings
4
5 import os
6 load_dotenv(override=True)
7
8 embedding_model = init_embeddings(
9     model="openai:text-embedding-3-large",
10     api_key=os.getenv("CLOSEAI_API_KEY"),

```

```

11     base_url=os.getenv("CLOSEAI_BASE_URL"),
12 )
13
14 index_config = {
15     "embed": embedding_model,
16     "dims": 3072,
17     "fields": ["$"]
18 }
19 store = InMemoryStore(
20     index = index_config
21 )
22
23 namespace1 = ("users", "Alice", "memories")
24 key1 = 'preferences'
25 value1 = {
26     "course": "计算机组成原理",
27     "sports": "跑步",
28     "food": "紫光园奶皮子酸奶"
29 }
30
31 namespace2 = ("users", "Bob", "memories")
32 key2 = 'preferences'
33 value2 = {
34     "course": "数字电路与模拟电路",
35     "sports": "跑步",
36     "food": "奶皮子糖葫芦"
37 }
38
39 namespace3 = ("users", "Black", "memories")
40 key3 = 'preferences'
41 value3 = {
42     "course": "数字电路与模拟电路",
43     "sports": "羽毛球",
44     "food": "紫光园奶皮子酸奶"
45 }
46
47 store.put(namespace1, key1, value1)
48 store.put(namespace2, key2, value2)
49 store.put(namespace3, key3, value3)

```

```

1 for item in store.search(("users", ), query="数电模电"):
2     print(item)

```

输出

```

1 Item(namespace=['users', 'Bob', 'memories'], key='preferences', value=
  {'course': '数字电路与模拟电路', 'sports': '跑步', 'food': '奶皮子糖葫芦'},
  created_at='2026-06-11T15:45:13.161389+00:00', updated_at='2026-06-
  11T15:45:13.161391+00:00', score=0.22494289397943232)
2 Item(namespace=['users', 'Black', 'memories'], key='preferences', value=
  {'course': '数字电路与模拟电路', 'sports': '羽毛球', 'food': '紫光园奶皮子酸
  奶'}, created_at='2026-06-11T15:45:13.748504+00:00', updated_at='2026-06-
  11T15:45:13.748506+00:00', score=0.21367212817763243)
3 Item(namespace=['users', 'Alice', 'memories'], key='preferences', value=
  {'course': '计算机组成原理', 'sports': '跑步', 'food': '紫光园奶皮子酸奶'},
  created_at='2026-06-11T15:45:12.599571+00:00', updated_at='2026-06-
  11T15:45:12.599573+00:00', score=0.1253981117634029)

```

注意：如果只是指定了 `query`，返回的是所有 `namespace` 前缀满足要求的 `item`。底层会按照向量相似度计算查询和候选的 `score`，输出结果按照 `score` 降序排列。可以结合 `limit` 或 `filter` 限制数据条数。这里不再演示。

3.3 在Agent运行图中访问长期记忆

我们可以在工具或中间件中访问长期记忆。

3.3.1 在工具中访问长期记忆

① 基于InMemoryStore

```

1 from langchain.chat_models import init_chat_model
2 from dotenv import load_dotenv
3 import os
4
5 # 从.env文件中加载环境变量
6 load_dotenv(override=True)
7
8 model = init_chat_model(
9     model="gpt-5.4-mini",
10    model_provider="openai",
11    api_key=os.getenv("CLOSEAI_API_KEY"),
12    base_url=os.getenv("CLOSEAI_BASE_URL")
13 )

```

```

1 from langchain_core.messages import HumanMessage
2 from typing import NotRequired
3
4 from langchain.agents import create_agent, AgentState
5 from langchain.tools import tool, ToolRuntime
6 from langgraph.store.memory import InMemoryStore
7
8 store = InMemoryStore()
9
10 class CustomState(AgentState):
11     user_id: NotRequired[str]
12
13 @tool(parse_docstring=True)
14 def save_user_info(name: str, runtime: ToolRuntime) -> str:
15     """
16     将用户信息保存在长期记忆中
17

```

```

18     Args:
19         name: 用户名
20
21     Returns:
22         str: 保存状态
23     """
24     runtime.store.put(("users",), runtime.state["user_id"], {"name": name})
25     return "saved"
26
27 @tool(parse_docstring=True)
28 def get_user_info(runtime: ToolRuntime) -> str:
29     """
30     从长期记忆中读取用户信息
31
32     Returns:
33         str: 用户信息
34     """
35     item = runtime.store.get(("users",), runtime.state["user_id"])
36     return str(item.value) if item else "unknown"
37
38 agent = create_agent(
39     model=model,
40     tools=[save_user_info, get_user_info],
41     store=store,
42     system_prompt="用户提及个人信息时及时记录，用户询问个人信息时尝试用工具检索",
43     state_schema=CustomState,
44 )
45
46 print("=" * 30, '-> 第一个会话（线程） <-', "=" * 30)
47 response1 = agent.invoke({
48     "messages": [HumanMessage("你好，很高兴认识你，我是小花")],
49     "user_id": "user-1"
50 })
51 for msg in response1["messages"]:
52     msg.pretty_print()
53
54 print("=" * 30, '-> 第二个会话（线程） <-', "=" * 30)
55 response2 = agent.invoke({
56     "messages": [HumanMessage("我是谁")],
57     "user_id": "user-1"
58 })
59 for msg in response2["messages"]:
60     msg.pretty_print()

```

其中，CustomState扩展了 Agent 的标准状态。除了默认的 `messages`（历史消息列表）之外，还额外增加了一个 `user_id` 字段。这样，Agent 在运行过程中随时随地都能知道当前和它说话的用户 ID 是什么。

输出

```

1  ===== -> 第一个会话（线程） <-
2  =====
3  ===== Human Message
4  =====
5  你好，很高兴认识你，我是小花

```



```

5  ===== Ai Message
   =====
6  Tool Calls:
7  save_user_info (call_dUFA22P95tnIrBc0fi2I0WLF)
8  Call ID: call_dUFA22P95tnIrBc0fi2I0WLF
9  Args:
10     name: 小花
11  ===== Tool Message
   =====
12  Name: save_user_info
13
14  saved
15  ===== Ai Message
   =====
16
17  你好, 小花, 很高兴认识你!
18  ===== -> 第二个会话(线程) <-
   =====
19  ===== Human Message
   =====
20
21  我是谁
22  ===== Ai Message
   =====
23  Tool Calls:
24  get_user_info (call_cARiaT5NR0kUZDbMopy0khqA)
25  Call ID: call_cARiaT5NR0kUZDbMopy0khqA
26  Args:
27  ===== Tool Message
   =====
28  Name: get_user_info
29
30  {'name': '小花'}
31  ===== Ai Message
   =====
32
33  你是小花。

```

两次invoke没有通过config串联为起来，是两个独立的会话，但第二个会话可以访问第一个会话写入长期记忆的内容。

② 基于PostgresStore

```

1  from typing import NotRequired
2
3  from langchain.agents import create_agent, AgentState
4  from langchain.tools import tool, ToolRuntime
5  from langgraph.store.postgres import PostgresStore
6
7  DB_URI =
8      "postgresql://langchain_user:abcd1234@118.195.128.47:5432/langchain_db?
9      sslmode=disable"
10
11  class CustomState(AgentState):
12      user_id: NotRequired[str]
13
14  @tool(parse_docstring=True)

```

```

13 def save_user_info(name: str, runtime: ToolRuntime) -> str:
14     """
15     将用户信息保存在长期记忆中
16
17     Args:
18         name: 用户名
19
20     Returns:
21         str: 保存状态
22     """
23     runtime.store.put(("users",), runtime.state["user_id"], {"name": name})
24     return "saved"
25
26 @tool(parse_docstring=True)
27 def get_user_info(runtime: ToolRuntime) -> str:
28     """
29     从长期记忆中读取用户信息
30
31     Returns:
32         str: 用户信息
33     """
34     item = runtime.store.get(("users",), runtime.state["user_id"])
35     return str(item.value) if item else "unknown"
36
37 with PostgresStore.from_conn_string(DB_URI) as store:
38     store.setup()
39     agent = create_agent(
40         model=model,
41         tools=[save_user_info, get_user_info],
42         store=store,
43         system_prompt="用户提及个人信息时及时记录，用户询问个人信息时尝试用工具检索",
44         state_schema=CustomState,
45     )
46
47     print("=" * 30, '-> 第一个会话（线程） <-', "=" * 30)
48     response1 = agent.invoke({
49         "messages": "你好，很高兴认识你，我是小花",
50         "user_id": "user-1"
51     })
52     for msg in response1["messages"]:
53         msg.pretty_print()
54
55     print("=" * 30, '-> 第二个会话（线程） <-', "=" * 30)
56     response2 = agent.invoke({
57         "messages": "我是谁",
58         "user_id": "user-1"
59     })
60     for msg in response2["messages"]:
61         msg.pretty_print()

```

输出

```

1  ===== -> 第一个会话（线程） <-
   =====
2  ===== Human Message
   =====
3

```

```

4  你好，很高兴认识你，我是小花
5  ===== Ai Message
6  =====
7  Tool Calls:
8  save_user_info (call_CmfPqf4aahyt5LQIgcaCSTDy)
9  Call ID: call_CmfPqf4aahyt5LQIgcaCSTDy
10 Args:
11   name: 小花
12 ===== Tool Message
13 =====
14 Name: save_user_info
15 saved
16 ===== Ai Message
17 =====
18 你好，小花，很高兴认识你！有什么我可以帮你的吗？
19 ===== -> 第二个会话（线程） <-
20 =====
21 ===== Human Message
22 =====
23 我是谁
24 ===== Ai Message
25 =====
26 Tool Calls:
27 get_user_info (call_x6DPJGLsqwjMgRoxd62bd7ZZ)
28 Call ID: call_x6DPJGLsqwjMgRoxd62bd7ZZ
29 Args:
30 ===== Tool Message
31 =====
32 Name: get_user_info
33 {'name': '小花'}
34 ===== Ai Message
35 =====
36 你是小花。

```

查看PostgreSQL数据表

打开Xshell客户端

```

1  ubuntu@vm-0-6-ubuntu:~$ psql
   postgresql://langgraph_user:123456@localhost:5432/langgraph_db?
   sslmode=disable
2  psql (16.13 (Ubuntu 16.13-0ubuntu0.24.04.1))
3  Type "help" for help.
4
5  langgraph_db=>

```

查看表

```

1 langgraph_db=> \dt
2                               List of relations
3 Schema |          Name          | Type |   Owner
4 -----+-----+-----+-----
5 public | checkpoint_blobs       | table | langgraph_user
6 public | checkpoint_migrations | table | langgraph_user
7 public | checkpoint_writes      | table | langgraph_user
8 public | checkpoints            | table | langgraph_user
9 public | store                  | table | langgraph_user
10 public | store_migrations       | table | langgraph_user
11 (6 rows)
12
13 langgraph_db=>

```

新增两张Store相关的表 `store` 和 `store_migrations`。

3.3.2 在中间件中访问长期记忆

① Node-style hooks中访问

以 `before_model` 为例，其钩子函数签名如下

```

1 def before_model(self, state: StateT, runtime: Runtime[ContextT]) ->
  dict[str, Any] | None:

```

`Runtime` 定义如下

```

1 @dataclass(**_DC_KWARGS)
2 class Runtime(Generic[ContextT]):
3     context: ContextT = field(default=None) # type: ignore[assignment]
4     """Static context for the graph run, like `user_id`, `db_conn`, etc.
5
6     Can also be thought of as 'run dependencies'."""
7
8     store: BaseStore | None = field(default=None)
9     """Store for the graph run, enabling persistence and memory."""
10
11     stream_writer: StreamWriter = field(default=_no_op_stream_writer)
12     """Function that writes to the custom stream."""
13
14     previous: Any = field(default=None)
15     """The previous return value for the given thread.
16
17     Only available with the functional API when a checkpointer is provided.
18     """
19     ...

```

所以，我们可以通过 `runtime.store` 在中间件中访问长期记忆。

② Wrap-style hooks中访问

1. wrap_model_call

钩子函数签名如下

```

1 def wrap_model_call(
2     self,
3     request: ModelRequest[ContextT],
4     handler: Callable[[ModelRequest[ContextT]], ModelResponse[ResponseT]],
5 ) -> ModelResponse[ResponseT] | AIMessage | ExtendedModelResponse[ResponseT]:

```

`ModelRequest` 定义如下

```

1 @dataclass(init=False)
2 class ModelRequest(Generic[ContextT]):
3     """Model request information for the agent.
4
5     Type Parameters:
6         ContextT: The type of the runtime context. Defaults to `None` if not
7         specified.
8
9     model: BaseChatModel
10    messages: list[AnyMessage] # excluding system message
11    system_message: SystemMessage | None
12    tool_choice: Any | None
13    tools: list[BaseTool | dict[str, Any]]
14    response_format: ResponseFormat[Any] | None
15    state: AgentState[Any]
16    runtime: Runtime[ContextT]
17    model_settings: dict[str, Any] = field(default_factory=dict)
18    ...

```

所以，可以通过 `request.runtime.store` 访问长期记忆。

2. wrap_tool_call

钩子函数签名如下

```

1 def wrap_tool_call(
2     self,
3     request: ToolCallRequest,
4     handler: Callable[[ToolCallRequest], ToolMessage | Command[Any]],
5 ) -> ToolMessage | Command[Any]:

```

`ToolCallRequest` 定义如下

```

1 @dataclass
2 class ToolCallRequest:
3     """Tool execution request passed to tool call interceptors.
4
5     Attributes:
6         tool_call: Tool call dict with name, args, and id from model output.
7         tool: BaseTool instance to be invoked, or None if tool is not
8             registered with the `ToolNode`. When tool is `None`,
9             interceptors can
10             handle the request without validation. If the interceptor calls
11             `execute()`,
12             validation will occur and raise an error for unregistered tools.
13         state: Agent state (`dict`, `list`, or `BaseModel`).

```

```

12         runtime: LangGraph runtime context (optional, `None` if outside
runtime: LangGraph runtime context (optional, `None` if outside
graph).
13         """
14
15         tool_call: ToolCall
16         tool: BaseTool | None
17         state: Any
18         runtime: ToolRuntime
19         ...

```

`ToolRuntime` 定义如下

```

1 @dataclass
2 class ToolRuntime(_DirectlyInjectedToolArg, Generic[ContextT, StateT]):
3     state: StateT
4     context: ContextT
5     config: RunnableConfig
6     stream_writer: StreamWriter
7     tool_call_id: str | None
8     store: BaseStore | None

```

所以，可以通过 `request.runtime.store` 访问长期记忆。

3.4 何时写入记忆

官方介绍了两种方式。

1. 在主流程里写 (hot path)

也就是：用户发消息，AI 一边回答，一边决定要不要记下来。

优点：

- 立即生效
- 下一轮马上能用
- 用户可感知，透明

缺点：

- 增加延迟
- 逻辑变复杂

2. 在后台写 (background)

就是先回答用户，记忆整理放到后台 `异步` 做。

优点：

- 主流程更快
- 记忆逻辑更独立
- 更适合批量整理

缺点：

- 不能立刻生效
- 要决定多久整理一次
- 触发时机不好选

工程上通常这么选：

- 用户偏好、账号资料：可热路径写
- 对话摘要、经验沉淀、行为分析：更适合后台写

4、课后阅读：Static runtime Context

静态运行时上下文(Static runtime Context)表示不可变的数据，如用户元数据、工具和传递给应用程序的数据库连接对象。通常在运行开始时通过 `invoke` / `stream` 的 `context` 参数传递，此类数据在运行期间不会更改。

4.1 中间件中访问

4.1.1 Node-style hooks

此类钩子都通过 `runtime.context` 访问上下文对象。

用户自定义 `ContextSchema` 用 `@dataclass` 修饰，在Agent创建时通过 `state_schema` 参数传递。

本例从长期记忆中查询用户额度，如果额度用尽则中断流程。

代码

```
1 from dataclasses import dataclass
2 from typing import Any
3
4 from langchain.agents import create_agent
5 from langchain.agents.middleware import AgentMiddleware, AgentState,
hook_config
6 from langchain.messages import AIMessage
7
8 from langgraph.runtime import Runtime
9 from langgraph.store.memory import InMemoryStore
10 from dotenv import load_dotenv
11 from loguru import logger
12
13 load_dotenv()
14
15 store = InMemoryStore()
16 namespace = ("users", "credits")
17 key1 = 'Ada Lovelace'
18 value1 = {
19     "tokens_credit_left": 5000,
20     "user_level": 5
21 }
22 key2 = 'Blackwell'
23 value2 = {
24     "tokens_credit_left": 2999,
25     "user_level": 5
26 }
27 key3 = 'Ampere'
28 value3 = {
29     "tokens_credit_left": 1000,
30     "user_level": 5
31 }
32 store.put(namespace, key1, value1)
33 store.put(namespace, key2, value2)
34 store.put(namespace, key3, value3)
35
36 @dataclass
```

```

37 class UserContext:
38     username: str
39
40 class CheckCredit(AgentMiddleware):
41     @hook_config(can_jump_to=["end"])
42     def before_model(self, state: AgentState, runtime: Runtime) ->
dict[str, Any] | None:
43         store = runtime.store
44         context = runtime.context
45
46         username = context.username
47         credit = store.get(namespace, username)
48         if not credit:
49             return {
50                 "jump_to": "end",
51                 "messages": AIMessage("您尚未注册~")
52             }
53
54         if credit.value["tokens_credit_left"] < 3000:
55             return {
56                 "jump_to": "end",
57                 "messages": AIMessage(f"{username}额度不足，请充值")
58             }
59
60         return None
61
62     def after_model(self, state: AgentState, runtime: Runtime) -> dict[str,
Any] | None:
63         store = runtime.store
64         context = runtime.context
65
66         username = context.username
67         credit = store.get(namespace, username)
68         usage_metadata = state["messages"][-1].usage_metadata
69         token_usage = usage_metadata["input_tokens"] +
usage_metadata["output_tokens"] * 6
70         credit.value["tokens_credit_left"] -= token_usage
71
72         # 更新长期记忆
73         store.put(namespace, username, credit.value)
74
75         return None
76
77     def after_agent(self, state: AgentState, runtime: Runtime) -> dict[str,
Any] | None:
78         store = runtime.store
79         context = runtime.context
80
81         username = context.username
82         item = store.get(namespace, username)
83         credit = item.value["tokens_credit_left"]
84
85         logger.info(f"{username} 当前剩余额度: {credit}")
86         return None
87
88
89 agent = create_agent(
90     model="deepseek-chat",

```



```

91     middleware=[CheckCredit()],
92     store=store,
93     context_schema=UserContext
94 )
95
96 response1 = agent.invoke(
97     {"messages": ["你好啊，你知道 Ada Lovelace 的贡献吗? "]},
98     context=UserContext(username="Ada Lovelace")
99 )
100 print("=" * 30, '-> Ada Lovelace messages <-', '=' * 30)
101 for msg in response1["messages"]:
102     msg.pretty_print()
103
104 response2 = agent.invoke(
105     {"messages": ["你好啊，你知道 Blackwell 的贡献吗? "]},
106     context=UserContext(username="Blackwell")
107 )
108 print("=" * 30, '-> Blackwell messages <-', '=' * 30)
109 for msg in response2["messages"]:
110     msg.pretty_print()

```

输出

```

1  •[32m2026-04-14 11:39:18.876•[0m | •[1mINFO      •[0m |
   •[36m__main__•[0m:•[36mafter_agent•[0m:•[36m85•[0m - •[1mAda Lovelace
   当前剩余额度: 2721•[0m
2  •[32m2026-04-14 11:39:18.881•[0m | •[1mINFO      •[0m |
   •[36m__main__•[0m:•[36mafter_agent•[0m:•[36m85•[0m - •[1mBlackwell 当前
   剩余额度: 2999•[0m
3
4  ===== -> Ada Lovelace messages <-
   =====
5  =====•[1m Human Message
   •[0m=====
6
7  你好啊，你知道 Ada Lovelace 的贡献吗?
8  =====•[1m Ai Message
   •[0m=====
9
10 是的，Ada Lovelace（1815-1852）被广泛认为是世界上第一位计算机程序员，她的贡献主要
   体现在以下几个方面：
11
12  ---
13
14  ### **1. 翻译与注释：解析“分析机”的潜力**
15  - 1843年，她翻译了意大利工程师路易吉·费德里科·米纳布里关于查尔斯·巴贝奇“分析机”（早
   期机械通用计算机）的论文，并添加了大量自己的注释，篇幅远超原文。
16  - 她的注释中包含了**世界上第一个计算机算法**，用于计算伯努利数，这被视为最早的“计算
   机程序”概念。
17
18  ---
19
20  ### **2. 超越计算的前瞻性思想**
21  - 她指出分析机不仅限于数值计算，还能处理符号、音乐甚至艺术，预见了计算机的**通用性**
   和**创造性潜力**。
22  - 她强调“机器只能执行指令，不能自主思考”，区分了人工智能与人类智能的界限。
23

```

```

24 ---
25
26 ### **3. 技术与人文的桥梁**
27 - 作为诗人拜伦的女儿，她将数学逻辑与想象力结合，主张科技与艺术的融合，这一思想对后世
  计算科学影响深远。
28
29 ---
30
31 ### **4. 历史意义**
32 - 尽管分析机在她有生之年未能建成，但她的工作在一个世纪后启发了艾伦·图灵等计算机科学先
  驱。
33 - 1980年，美国国防部将一种编程语言命名为“Ada”以纪念她。
34
35 ---
36
37 ### **争议与澄清**
38 - 部分学者认为她的贡献被过度神话，但主流观点仍肯定她首次系统阐述了编程逻辑的核心思
  想，尤其在19世纪女性科学参与受限的背景下，她的成就尤为突出。
39
40 ---
41
42 Ada Lovelace 的故事不仅是技术史上的里程碑，也象征着跨学科创新如何推动人类认知边界。
  如果需要更具体的细节，我可以进一步展开！
43 ===== -> Blackwell messages <-
  =====
44 =====•[1m Human Message
  •[0m=====
45
46 你好啊，你知道 Blackwell 的贡献吗？
47 =====•[1m Ai Message
  •[0m=====
48
49 Blackwell额度不足，请充值

```

4.1.2 Wrap-style hooks

① wrap_model_call

本例通过 `context` 中记录的用户身份信息调整暴露给模型的工具集。

此处更改模型请求的方法是 `transient request update`，见上文[短期记忆 2-4-2](#)

```

1 from dataclasses import dataclass
2 from typing import Callable
3
4 from langchain.agents import create_agent
5 from langchain.agents.middleware import AgentMiddleware, ModelResponse,
  ModelRequest
6 from langchain.messages import AIMessage
7 from langchain.tools import tool
8
9 from langgraph.store.memory import InMemoryStore
10 from dotenv import load_dotenv
11 from loguru import logger
12
13 load_dotenv()
14
15 store = InMemoryStore()

```

```

16 namespace = ("users", "credits")
17 key1 = 'Ada Lovelace'
18 value1 = {
19     "tokens_left": 3000, # 剩余大模型token额度
20     "get_weather": "yes", # 是否认证天气查询服务
21     "get_news": "no" # 是否认证网络爬虫服务，用于查询新闻
22 }
23 key2 = 'Blackwell'
24 value2 = {
25     "tokens_left": 2000, # 剩余大模型token额度
26     "get_weather": "no", # 是否认证天气查询服务
27     "get_news": "yes" # 是否认证网络爬虫服务，用于查询新闻
28 }
29 key3 = 'Ampere'
30 value3 = {
31     "tokens_left": 3000, # 剩余大模型token额度
32     "get_weather": "yes", # 是否认证天气查询服务
33     "get_news": "yes" # 是否认证网络爬虫服务，用于查询新闻
34 }
35 store.put(namespace, key1, value1)
36 store.put(namespace, key2, value2)
37 store.put(namespace, key3, value3)
38
39 @tool
40 def get_weather(city: str):
41     """
42     查询指定城市当日天气
43
44     Args:
45         city: 城市名称
46     """
47     return f"{city} 今天天气不错"
48
49 @tool()
50 def get_news():
51     """查询当日新闻"""
52     return "美伊尚未达成停战协议"
53
54 @dataclass
55 class UserContext:
56     username: str
57
58 class CheckCredit(AgentMiddleware):
59     def wrap_model_call(
60         self,
61         request: ModelRequest,
62         handler: Callable[[ModelRequest], ModelResponse],
63     ) -> ModelResponse:
64         context = request.runtime.context
65         store = request.runtime.store
66
67         value = store.get(namespace, context.username).value
68         tools = []
69
70         for tool in request.tools:
71             tool_name = tool.name
72             if value[tool_name] == "yes":
73                 tools.append(tool)

```

```

74         else:
75             logger.warning(f"{context.username} 无权调用 {tool_name}")
76
77         # 更改调用请求携带的工具集，仅本次生效
78         request = request.override(tools=tools)
79
80         return handler(request)
81
82 agent = create_agent(
83     model="deepseek-chat",
84     middleware=[CheckCredit()],
85     tools=[get_weather, get_news],
86     store=store,
87     context_schema=UserContext
88 )
89
90 response1 = agent.invoke(
91     {"messages": ["你好啊，今天北京天气如何，今日有哪些新闻？"]},
92     context=UserContext(username="Ada Lovelace")
93 )
94 print("=" * 30, '-> Ada Lovelace messages <-', '=' * 30)
95 for msg in response1["messages"]:
96     msg.pretty_print()
97
98 response2 = agent.invoke(
99     {"messages": ["你好啊，今天北京天气如何，今日有哪些新闻？"]},
100    context=UserContext(username="Blackwell")
101 )
102 print("=" * 30, '-> Blackwell messages <-', '=' * 30)
103 for msg in response2["messages"]:
104     msg.pretty_print()
105
106 response3 = agent.invoke(
107     {"messages": ["你好啊，今天北京天气如何，今日有哪些新闻？"]},
108     context=UserContext(username="Ampere")
109 )
110 print("=" * 30, '-> Ampere messages <-', '=' * 30)
111 for msg in response3["messages"]:
112     msg.pretty_print()

```

输出

```

1  •[32m2026-04-14 11:40:07.527•[0m | •[33m•[1mWARNING •[0m |
   •[36m__main__•[0m:•[36mwrap_model_call•[0m:•[36m75•[0m - •[33m•[1mAda
   Lovelace 无权调用 get_news•[0m
2  •[32m2026-04-14 11:40:11.357•[0m | •[33m•[1mWARNING •[0m |
   •[36m__main__•[0m:•[36mwrap_model_call•[0m:•[36m75•[0m - •[33m•[1mAda
   Lovelace 无权调用 get_news•[0m
3  •[32m2026-04-14 11:40:15.049•[0m | •[33m•[1mWARNING •[0m |
   •[36m__main__•[0m:•[36mwrap_model_call•[0m:•[36m75•[0m -
   •[33m•[1mBlackwell 无权调用 get_weather•[0m
4
5  ===== -> Ada Lovelace messages <-
   =====
6  =====•[1m Human Message
   •[0m=====
7

```

```
8  你好啊，今天北京天气如何，今日有哪些新闻？
9  =====[1m Ai Message
   •[0m=====
10
11 我来帮您查询北京的天气情况。不过关于今日新闻，我目前没有相关的查询功能。
12 Tool Calls:
13 get_weather (call_00_OpIicbLHGEaTanH3yaTDuiMp)
14 Call ID: call_00_OpIicbLHGEaTanH3yaTDuiMp
15 Args:
16   city: 北京
17 =====[1m Tool Message
   •[0m=====
18 Name: get_weather
19
20 北京 今天天气不错
21 =====[1m Ai Message
   •[0m=====
22
23 根据查询结果，北京今天天气不错！
24
25 关于今日新闻，我目前无法为您提供实时的新闻信息。您可以：
26 1. 查看各大新闻网站或新闻应用
27 2. 关注官方新闻发布平台
28 3. 使用专门的新闻查询工具
29
30 如果您需要了解其他城市的天气情况，我很乐意为您查询。
31
32 •[32m2026-04-14 11:40:17.903•[0m | •[33m•[1mWARNING •[0m |
   •[36m__main__•[0m:•[36mwrap_model_call•[0m:•[36m75•[0m -
   •[33m•[1mBlackwell 无权调用 get_weather•[0m
33
34 ===== -> Blackwell messages <-
   =====
35 =====[1m Human Message
   •[0m=====
36
37 你好啊，今天北京天气如何，今日有哪些新闻？
38 =====[1m Ai Message
   •[0m=====
39
40 我目前无法查询天气信息，不过我可以帮您查询今天的新闻。让我为您获取一下今天的新闻内容。
41 Tool Calls:
42   get_news (call_00_thc8aLWFXkeffY1vafIwMJLk)
43   Call ID: call_00_thc8aLWFXkeffY1vafIwMJLk
44   Args:
45 =====[1m Tool Message
   •[0m=====
46 Name: get_news
47
48 美伊尚未达成停战协议
49 =====[1m Ai Message
   •[0m=====
50
51 根据查询结果，今天有一条重要新闻：**美伊尚未达成停战协议**。
52
53 关于北京的天气情况，我目前无法为您提供实时天气信息。建议您可以通过以下方式获取准确的北京天气信息：
54 1. 查看天气预报应用或网站
```

```

55 2. 使用搜索引擎搜索"北京天气"
56 3. 查看手机自带的天气应用
57
58 如果您还想了解其他新闻或者有其他问题，我很乐意为您提供帮助！
59 ===== -> Ampere messages <-
60 =====
61 =====•[1m Human Message
62 •[0m=====
63
64 你好啊，今天北京天气如何，今日有哪些新闻？
65 =====•[1m Ai Message
66 •[0m=====
67
68 我来帮您查询北京的天气和今日新闻。
69 Tool Calls:
70   get_weather (call_00_ZyVpPMawPgdfdv49ojD3zIFt)
71   Call ID: call_00_ZyVpPMawPgdfdv49ojD3zIFt
72   Args:
73     city: 北京
74   get_news (call_01_kZaouxjSdhjrJGmzGWRbP3so)
75   Call ID: call_01_kZaouxjSdhjrJGmzGWRbP3so
76   Args:
77 =====•[1m Tool Message
78 •[0m=====
79 Name: get_weather
80
81 北京 今天天气不错
82 =====•[1m Tool Message
83 •[0m=====
84 Name: get_news
85
86 美伊尚未达成停战协议
87 =====•[1m Ai Message
88 •[0m=====
89
90 根据查询结果：
91
92 **北京天气：**
93 今天北京天气不错。
94
95 **今日新闻：**
96 - 美伊尚未达成停战协议
97
98 如果您需要更详细的天气信息（比如具体温度、湿度等）或者想了解其他新闻，请告诉我！

```

② wrap_tool_call

在上述案例的基础上，我们补充额度校验，和长期记忆更新。

```

1  from dataclasses import dataclass
2  from typing import Callable, Any
3
4  from langchain.agents import create_agent
5  from langchain.agents.middleware import AgentMiddleware, ModelResponse,
   ModelRequest, ToolCallRequest
6  from langchain.messages import AIMessage, ToolMessage
7  from langchain.tools import tool

```

```

8
9 from langgraph.store.memory import InMemoryStore
10 from langgraph.types import Command
11 from dotenv import load_dotenv
12 from loguru import logger
13
14 load_dotenv()
15
16 store = InMemoryStore()
17 namespace = ("users", "credits")
18 key1 = 'Ada Lovelace'
19 value1 = {
20     "tokens_left": 3000, # 剩余大模型token额度
21     "get_weather": "yes", # 是否认证天气查询服务
22     "get_news": "no" # 是否认证网络爬虫服务，用于查询新闻
23 }
24 key2 = 'Blackwell'
25 value2 = {
26     "tokens_left": 2000, # 剩余大模型token额度
27     "get_weather": "no", # 是否认证天气查询服务
28     "get_news": "yes" # 是否认证网络爬虫服务，用于查询新闻
29 }
30 key3 = 'Ampere'
31 value3 = {
32     "tokens_left": 3000, # 剩余大模型token额度
33     "get_weather": "yes", # 是否认证天气查询服务
34     "get_news": "yes" # 是否认证网络爬虫服务，用于查询新闻
35 }
36 store.put(namespace, key1, value1)
37 store.put(namespace, key2, value2)
38 store.put(namespace, key3, value3)
39
40 # 工具额度消耗
41 CREDIT_MAP = {
42     "get_weather": 10,
43     "get_news": 20
44 }
45
46 @tool
47 def get_weather(city: str):
48     """
49     查询指定城市当日天气
50
51     Args:
52         city: 城市名称
53     """
54     return f"{city} 今天天气不错"
55
56 @tool()
57 def get_news():
58     """查询当日新闻"""
59     return "美伊尚未达成停战协议"
60
61 @dataclass
62 class UserContext:
63     username: str
64
65 class CheckCredit(AgentMiddleware):

```

```

66     def wrap_model_call(
67         self,
68         request: ModelRequest,
69         handler: Callable[[ModelRequest], ModelResponse],
70     ) -> ModelResponse:
71         context = request.runtime.context
72         store = request.runtime.store
73
74         value = store.get(namespace, context.username).value
75         tools = []
76
77         for tool in request.tools:
78             tool_name = tool.name
79             if value[tool_name] == "yes":
80                 tools.append(tool)
81             else:
82                 logger.warning(f"{context.username} 无权调用 {tool_name}")
83
84         # 更改调用请求携带的工具集, 仅本次生效
85         request = request.override(tools=tools)
86
87         return handler(request)
88
89 class ToolGuard(AgentMiddleware):
90     def wrap_tool_call(
91         self,
92         request: ToolCallRequest,
93         handler: Callable[[ToolCallRequest], ToolMessage | Command[Any]],
94     ) -> ToolMessage | Command[Any]:
95         store = request.runtime.store
96         context = request.runtime.context
97
98         username = context.username
99         value = store.get(namespace, username).value
100         tool_name = request.tool.name
101
102         # 本次调用花费
103         credits_cost = CREDIT_MAP.get(tool_name)
104         # 剩余额度
105         credits_left = value["tokens_left"]
106
107         if credits_left < credits_cost:
108             logger.warning(f"{username} 额度不足, {tool_name} 调用失败!")
109             return Command(
110                 update={
111                     "messages": [
112                         ToolMessage(
113                             content="额度不足, 无法调用工具, 请充值~",
114                             tool_call_id=request.runtime.tool_call_id
115                         )
116                     ]
117                 }
118             )
119
120         # 更新剩余额度
121         credits_left -= credits_cost
122         value["tokens_left"] = credits_left
123         store.put(namespace, username, value)

```



```

124     logger.info(f"{username} 调用 {tool_name} 花费 {credits_cost}, 剩余额
    度 {credits_left}")
125     logger.info(f"{username} 状态已更新, 当前状态 {store.get(namespace,
    username).value}")
126
127     return handler(request)
128
129 agent = create_agent(
130     model="deepseek-chat",
131     middleware=[CheckCredit(), ToolGuard()],
132     tools=[get_weather, get_news],
133     store=store,
134     context_schema=UserContext
135 )
136
137 response = agent.invoke(
138     {"messages": ["你好啊, 今天北京天气如何, 今日有哪些新闻?"]},
139     context=UserContext(username="Ada Lovelace")
140 )
141 print("=" * 30, '-> Ada Lovelace messages <-', '=' * 30)
142 for msg in response["messages"]:
143     msg.pretty_print()

```

输出

```

1  •[32m2026-04-14 11:41:20.386•[0m | •[33m•[1mWARNING •[0m |
  •[36m__main__•[0m:•[36mwrap_model_call•[0m:•[36m82•[0m - •[33m•[1mAda
  Lovelace 无权调用 get_news•[0m
2  •[32m2026-04-14 11:41:23.859•[0m | •[1mINFO •[0m |
  •[36m__main__•[0m:•[36mwrap_tool_call•[0m:•[36m124•[0m - •[1mAda
  Lovelace 调用 get_weather 花费 10, 剩余额度 2990•[0m
3  •[32m2026-04-14 11:41:23.860•[0m | •[1mINFO •[0m |
  •[36m__main__•[0m:•[36mwrap_tool_call•[0m:•[36m125•[0m - •[1mAda
  Lovelace 状态已更新, 当前状态 {'tokens_left': 2990, 'get_weather': 'yes',
  'get_news': 'no'}•[0m
4  •[32m2026-04-14 11:41:23.865•[0m | •[33m•[1mWARNING •[0m |
  •[36m__main__•[0m:•[36mwrap_model_call•[0m:•[36m82•[0m - •[33m•[1mAda
  Lovelace 无权调用 get_news•[0m
5
6  ===== -> Ada Lovelace messages <-
  =====
7  =====•[1m Human Message
  •[0m=====
8
9  你好啊, 今天北京天气如何, 今日有哪些新闻?
10 =====•[1m Ai Message
    •[0m=====
11
12 我来帮您查询北京的天气情况。不过需要说明的是, 我目前只能查询天气信息, 无法提供新闻资
    讯。
13 Tool calls:
14 get_weather (call_00_HY2jENXsRyK21gITiQgevsk9)
15 Call ID: call_00_HY2jENXsRyK21gITiQgevsk9
16 Args:
17     city: 北京
18 =====•[1m Tool Message
    •[0m=====

```

```

19 Name: get_weather
20
21 北京 今天天气不错
22 =====[1m Ai Message
   •[0m=====
23
24 根据查询结果，北京今天天气不错。
25
26 关于新闻资讯，我目前没有相关的查询功能。如果您想了解今日新闻，建议您：
27 1. 查看主流新闻网站或APP
28 2. 关注官方新闻媒体
29 3. 使用专门的新闻搜索工具
30
31 如果您还需要了解其他城市的天气情况，我很乐意为您查询。

```

4.1.3 特殊的便捷中间件dynamic_prompt

如前所述，@dynamic_prompt底层实现机制和通用钩子函数是统一的，对于动态提示词处理提供了更加便捷的入口。

本例通过静态上下文中携带的用户信息，动态更改系统提示词。

```

1  from dataclasses import dataclass
2  from langchain.agents import create_agent
3  from langchain.agents.middleware import dynamic_prompt, ModelRequest
4
5  from langgraph.store.memory import InMemoryStore
6  from dotenv import load_dotenv
7  from loguru import logger
8
9  load_dotenv()
10
11  # 准备历史记忆
12  namespace = ("users", "preferences")
13  key1 = "Ada Lovelace"
14  value1 = {
15      "chat_preferences": [
16          "不喜欢啰嗦",
17          "尽可能用最精简的文字解释清楚"
18      ]
19  }
20
21  key2 = "Blackwell"
22  value2 = {
23      "chat_preferences": [
24          "喜欢长篇大论、引经据典",
25          "不知道的东西要明确说'不知道'，不能胡编乱造"
26      ]
27  }
28
29  store = InMemoryStore()
30  store.put(namespace, key1, value1)
31  store.put(namespace, key2, value2)
32
33  @dataclass
34  class UserContext:
35      username: str

```

```

36
37 @dynamic_prompt
38 def personalized_prompt(request: ModelRequest) -> str:
39     username = request.runtime.context.username
40     store = request.runtime.store
41
42     preferences = store.get(namespace, username).value["chat_preferences"]
43     custom_prompt = f"# 用户偏好\n{'\n'.join(preferences)}"
44     logger.info(f"{username} 自定义系统提示词\n{custom_prompt}")
45
46     return custom_prompt
47
48 agent = create_agent(
49     model="deepseek-chat",
50     middleware=[personalized_prompt],
51     store=store,
52     context_schema=UserContext
53 )
54
55 response1 = agent.invoke(
56     {"messages": ["为什么花儿这样红? "]},
57     context=UserContext(username="Ada Lovelace")
58 )
59 print("=" * 30, '-> Ada Lovelace messages <-', '=' * 30)
60 for msg in response1["messages"]:
61     msg.pretty_print()
62
63 response2 = agent.invoke(
64     {"messages": ["为什么花儿这样红? "]},
65     context=UserContext(username="Blackwell")
66 )
67 print("=" * 30, '-> Blackwell messages <-', '=' * 30)
68 for msg in response2["messages"]:
69     msg.pretty_print()

```

输出

```

1  •[32m2026-04-14 11:44:10.784•[0m | •[1mINFO      •[0m |
   •[36m__main__•[0m:•[36mpersonalized_prompt•[0m:•[36m44•[0m - •[1mAda
   Lovelace 自定义系统提示词
2  # 用户偏好
3  不喜欢啰嗦
4  尽可能用最精简的文字解释清楚•[0m
5  •[32m2026-04-14 11:44:13.622•[0m | •[1mINFO      •[0m |
   •[36m__main__•[0m:•[36mpersonalized_prompt•[0m:•[36m44•[0m -
   •[1mBlackwell 自定义系统提示词
6  # 用户偏好
7  喜欢长篇大论、引经据典
8  不知道的东西要明确说'不知道', 不能胡编乱造•[0m
9
10 ===== -> Ada Lovelace messages <-
   =====
11 =====•[1m Human Message
   •[0m=====
12
13 为什么花儿这样红?

```

=====[1m Ai Message
•[0m=====

花儿红的主要原因是花瓣细胞中的**花青素**在酸性环境下显红色，同时受光照、温度、养分等影响，为吸引传粉者进化而成。

====-> Blackwell messages <-
=====

=====[1m Human Message
•[0m=====

为什么花儿这样红？

=====[1m Ai Message
•[0m=====

“为什么花儿这样红”这一问题，可以从科学、文学、文化等多个维度展开探讨。以下将从不同角度进行详细分析，力求全面而深入。

一、科学视角：色彩形成的自然机制

花朵的红色主要源于植物色素、光学结构以及进化适应性的共同作用。

1. **色素基础：花青素的主导作用**

- 花青素（Anthocyanin）是形成红、紫、蓝等颜色的关键色素，属于类黄酮化合物。它在不同酸碱度（pH）下呈现颜色变化：酸性环境中偏红（如玫瑰、牡丹），中性时呈紫色，碱性环境中则偏蓝。
- 其他色素如类胡萝卜素（橙黄色）和甜菜色素（少数植物如仙人掌）也可能与红色相关，但花青素是最常见的红色来源。

2. **光学与结构增色**

- 部分花朵的红色并非单纯依赖色素，而是通过花瓣表面的微观结构（如多层薄膜、衍射光栅）反射特定波长的光，增强色彩饱和度。例如红罂粟的花瓣具有蜂窝状结构，能强化红色视觉效果。

3. **进化与生态学意义**

- **传粉者吸引**：红色对许多传粉昆虫（如蜜蜂）可见度较低，但对鸟类（如蜂鸟）和部分蝶类极具吸引力。在美洲、亚洲等地区，红色花朵常与鸟媒传粉相关。
- **环境适应**：花青素可抵御紫外线伤害、抵抗病原体，并在低温或干旱条件下保护植物细胞。高海拔地区的红花比例较高，可能与紫外线防护有关。

二、文学与艺术中的象征意义

红色在人类文化中被赋予丰富内涵，花朵的红色常成为情感与哲思的载体。

1. **爱情与激情**

- 红玫瑰在西方传统中象征炽热爱情，可追溯至希腊罗马神话中维纳斯与阿多尼斯的故事，玫瑰的红色被联想为爱神之血。波斯诗人萨迪在《蔷薇园》中亦以玫瑰隐喻激情。

2. **生命、牺牲与革命**

- 中国文学中，红色梅花、杜鹃常被赋予坚贞或壮烈的意象（如“杜鹃啼血”）。近现代以来，红色花朵亦与革命精神关联，如《红岩》中的红梅象征烈士鲜血。

3. **美学与哲学隐喻**

- 王阳明在《传习录》中提及“岩中花树”，探讨心与物的关系；歌德在《色彩论》中分析红色对情感的影响。红色花朵的视觉冲击力，常引发对生命短暂与绚烂的思考（如日本“物哀”美学中的樱花）。

```
55 ---
56
57 ### **三、文化差异与历史演变**
58 不同文明对红色花朵的解读存在显著差异：
59
60 1. **东方文化**
61     - 中国：红色象征吉祥、喜庆，牡丹被称为“国色天香”，红色牡丹尤其代表富贵。但李贺诗句“桃花乱落如红雨”亦暗含哀愁。
62     - 日本：红椿花在江户时代被赋予“谦逊”与“永恒”之意，但亦因整朵凋落的特点与武士精神关联。
63
64 2. **西方文化**
65     - 基督教艺术中，红色玫瑰象征殉道者的鲜血与神圣之爱（如但丁《神曲》中天堂的玫瑰）。
66     - 近代欧洲，红罂粟因一战战场盛放，成为纪念阵亡将士的符号（如约翰·麦克雷诗作《在弗兰德战场》）。
67
68 ---
69
70 ### **四、现代科学的新发现**
71 近年研究进一步揭示了花朵红色的复杂性：
72     - **基因调控**：MYB转录因子等基因可精确控制花青素合成，通过基因工程已培育出深浅各异的红色花卉。
73     - **气候响应**：全球变暖可能导致花朵颜色加深，因高温促进花青素积累（2020年《生态学通讯》研究）。
74     - **传粉者视觉模型**：蜂鸟对红色的偏好与其视网膜中特殊色素细胞（油滴滤光）相关，这解释了美洲红色花与蜂鸟的协同进化。
75
76 ---
77
78 ### **结语**
79 “花儿这样红”既是自然选择的精巧结果，也是人类文明的情感投射。从花青素的生物化学路径到《诗经》中的“桃之夭夭”，从达尔文对传粉生态的观察到现代基因图谱的解析，这一问题串联起了科学与人文的对话。红色花朵的绚丽，最终在观察者的眼中完成了自然法则与心灵共鸣的统一。
80
81 **参考文献方向**（如需进一步研究可查阅）：
82     - 植物生理学：《植物生物化学与分子生物学》（B.B. Buchanan等编著）
83     - 文化象征：《花的语言》（Jack Goody，人类学著作）
84     - 进化生态学：《花的起源与多样性》（昆虫传粉协同进化理论）
```

dynamic_prompt 是动态更新本次调用前的系统提示词，并不会更改消息列表，所以根据模型输出，可以发现系统提示词生效，但消息列表看不到相关提示词。

4.2 工具中访问

在上文介绍工具的自定义args_schema时我们介绍了两种方式：

- 基于pydantic模型
- 基于json_schema

实际上，Langchain底层在生成工具的schema时，如果参数类型注解是pydantic模型，也会被一并解析。

```
1 from pydantic import BaseModel, Field
2 from typing import List
3
4 from langchain.tools import tool
```

```

5 from langgraph.prebuilt.tool_node import ToolRuntime
6 from langchain_core.utils.function_calling import convert_to_openai_tool
7
8 class UserInfo(BaseModel):
9     username: str = Field(description="用户名")
10    age: int = Field(description="年龄")
11    hobbies: List[str] = Field(description="兴趣爱好")
12
13 @tool(parse_docstring=True)
14 def save_user_info(user_info: UserInfo, runtime: ToolRuntime) -> str:
15     """
16     保存用户信息
17
18     Args:
19         user_info: 用户信息对象
20     """
21     return "ok"
22
23 print(convert_to_openai_tool(save_user_info))

```

输出如下

```

1  {
2      "type": "function",
3      "function": {
4          "name": "save_user_info",
5          "description": "保存用户信息",
6          "parameters": {
7              "properties": {
8                  "user_info": {
9                      "properties": {
10                         "username": {
11                             "description": "用户名",
12                             "type": "string"
13                         },
14                         "age": {
15                             "description": "年龄",
16                             "type": "integer"
17                         },
18                         "hobbies": {
19                             "description": "兴趣爱好",
20                             "items": {
21                                 "type": "string"
22                             },
23                             "type": "array"
24                         }
25                     },
26                     "required": [
27                         "username",
28                         "age",
29                         "hobbies"
30                     ],
31                     "type": "object",
32                     "description": "用户信息对象"
33                 }
34             },
35             "required": [

```

```

36         "user_info"
37     ],
38     "type": "object"
39 }
40 }
41 }

```

下面的案例中，我们通过工具读写长期记忆中保存的用户信息

```

1  from pydantic import BaseModel, Field
2  from typing import List, Any
3  from dataclasses import dataclass
4
5  from langchain.agents import create_agent
6  from langchain.tools import tool
7  from langchain.messages import HumanMessage, SystemMessage
8
9  from langgraph.prebuilt.tool_node import ToolRuntime
10 from langgraph.store.memory import InMemoryStore
11 from langgraph.checkpoint.memory import InMemorySaver
12
13 from loguru import logger
14 from dotenv import load_dotenv
15 load_dotenv()
16
17 @dataclass
18 class UserContext:
19     user_id: str
20
21 store = InMemoryStore()
22 namespace = ("users", "user_info")
23
24 class UserInfo(BaseModel):
25     username: str = Field(description="用户名", default="unknown")
26     age: int = Field(description="年龄", default=0)
27     # 这里相当于告诉模型，每次创建新实例时，调用list()创建一个新的list传进去
28     hobbies: List[str] = Field(description="兴趣爱好", default_factory=list)
29     # 这种写法是用可变对象做默认值，会有多个对象共享一个列表对象的风险
30     # hobbies: List[str] = Field(description="兴趣爱好", default=[])
31
32 @tool(parse_docstring=True)
33 def read_user_info(runtime: ToolRuntime[UserContext, Any]) -> UserInfo | str:
34     """
35     读取用户信息
36
37     Returns:
38         UserInfo | str: 如果找到用户信息，返回 UserInfo 对象；否则返回空字符串。
39     """
40     user_id = runtime.context.user_id
41     store = runtime.store
42     item = store.get(namespace, user_id)
43     if item:
44         return UserInfo(**item.value)
45     return ''
46
47 @tool(parse_docstring=True)

```

```

48 def write_user_info(user_info: UserInfo, runtime: ToolRuntime[UserContext,
Any]) -> bool:
49     """
50     将用户信息写入长期记忆
51
52     Args:
53         user_info: 用户信息对象
54
55     Returns:
56         bool: 如果成功写入则返回True, 否则返回False
57     """
58     user_id = runtime.context.user_id
59     store = runtime.store
60     try:
61         logger.info(f"写入记忆, namespace: {namespace}, user_id: {user_id},
user_info: {user_info.model_dump()}")
62         store.put(namespace, user_id, user_info.model_dump())
63     except Exception as e:
64         logger.error(e)
65         return False
66     return True
67
68 agent = create_agent(
69     model = "deepseek-chat",
70     tools = [read_user_info, write_user_info],
71     store = store,
72     checkpoint=InMemorySaver(),
73     context_schema=UserContext
74 )
75
76 config1 = {"configurable": {"thread_id": "thread_1"}}
77 config2 = {"configurable": {"thread_id": "thread_2"}}
78
79 agent.invoke(
80     {"messages": [
81         SystemMessage("如果输入包含用户信息，抽取并调用工具记录。在记录之前先查找历史
信息，和新增信息合并后记录。"
82             "如果用户提问设计个人信息，调用工具查找"),
83         HumanMessage("你好，我是韩立，我喜欢修仙")
84     ]},
85     context = UserContext(user_id = 'user_1'),
86     config = config1
87 )
88
89 thread_1_response = agent.invoke(
90     {"messages": ["我今年两百岁了，是你们口中的“元婴老怪”，我喜欢跑步"]},
91     context = UserContext(user_id = 'user_1'),
92     config = config1
93 )
94
95 print("=" * 30, '-> thread_1 messages <-', '=' * 30)
96 for msg in thread_1_response["messages"]:
97     msg.pretty_print()
98
99 thread_2_response = agent.invoke(
100     {"messages": ["你还记得我吗? "]},
101     context = UserContext(user_id = 'user_1'),
102     config = config2

```



```

103 )
104
105 print("=" * 30, '-> thread_2 messages <-', '=' * 30)
106 for msg in thread_2_response["messages"]:
107     msg.pretty_print()

```

输出

```

1  •[32m2026-04-14 15:58:30.011•[0m | •[1mINFO      •[0m |
  •[36m__main__•[0m:•[36mwrite_user_info•[0m:•[36m61•[0m - •[1m写入记忆,
  namespace: ('users', 'user_info'), user_id: user_1, user_info:
  {'username': '韩立', 'age': 0, 'hobbies': ['修仙']}]•[0m
2  •[32m2026-04-14 15:58:41.439•[0m | •[1mINFO      •[0m |
  •[36m__main__•[0m:•[36mwrite_user_info•[0m:•[36m61•[0m - •[1m写入记忆,
  namespace: ('users', 'user_info'), user_id: user_1, user_info:
  {'username': '韩立', 'age': 200, 'hobbies': ['修仙', '跑步']}]•[0m
3
4  ===== -> thread_1 messages <-
  =====
5  =====•[1m System Message
  •[0m=====
6
7  如果输入包含用户信息，抽取并调用工具记录。在记录之前先查找历史信息，和新增信息合并后记
  录。如果用户提问设计个人信息，调用工具查找
8  =====•[1m Human Message
  •[0m=====
9
10 你好，我是韩立，我喜欢修仙
11 =====•[1m Ai Message
  •[0m=====
12
13 你好韩立！很高兴认识你。我看到你喜欢修仙，这是一个很有趣的爱好。
14
15 为了让我能更好地记住你的信息，让我先查看一下是否已经有你的历史记录。
16 Tool Calls:
17 read_user_info (call_00_DaQBrMPFS4iZVK5F7mvaOsBy)
18 Call ID: call_00_DaQBrMPFS4iZVK5F7mvaOsBy
19 Args:
20 =====•[1m Tool Message
  •[0m=====
21 Name: read_user_info
22
23
24 =====•[1m Ai Message
  •[0m=====
25
26 看起来目前还没有你的历史记录。让我记录一下你的基本信息：
27 Tool Calls:
28 write_user_info (call_00_Zoz8sLHivXkA0a0Py2yvDFJB)
29 Call ID: call_00_Zoz8sLHivXkA0a0Py2yvDFJB
30 Args:
31   user_info: {'username': '韩立', 'hobbies': ['修仙']}
32 =====•[1m Tool Message
  •[0m=====
33 Name: write_user_info
34
35 true

```

```

36 =====•[1m Ai Message
37 •[0m=====
38 好的，我已经记录了你喜欢修仙这个爱好。修仙确实是一个很有深度的领域，涉及到修炼、功法、
39 丹药、法宝等很多方面。你对修仙的哪个方面特别感兴趣呢？比如是修炼功法、炼丹术、阵法，还
40 是其他方面？
41 =====•[1m Human Message
42 •[0m=====
43 我今年两百岁了，是你们口中的“元婴老怪”，我喜欢跑步
44 =====•[1m Ai Message
45 •[0m=====
46 哇！两百岁的元婴老怪，这真是令人惊叹！看来您在修仙道路上已经走了很长的路。
47 让我更新一下您的信息，包括年龄和新的爱好——跑步。
48 Tool Calls:
49 read_user_info (call_00_I9A03Y8RJSQLeHhwBGszkuiw)
50 Call ID: call_00_I9A03Y8RJSQLeHhwBGszkuiw
51 Args:
52 =====•[1m Tool Message
53 •[0m=====
54 Name: read_user_info
55 username='韩立' age=0 hobbies=['修仙']
56 =====•[1m Ai Message
57 •[0m=====
58 现在我来更新您的完整信息：
59 Tool Calls:
60 write_user_info (call_00_3JpZC48aqJG7m0m6X2EnZFNH)
61 Call ID: call_00_3JpZC48aqJG7m0m6X2EnZFNH
62 Args:
63 user_info: {'username': '韩立', 'age': 200, 'hobbies': ['修仙', '跑步']}
64 =====•[1m Tool Message
65 •[0m=====
66 Name: write_user_info
67 true
68 =====•[1m Ai Message
69 •[0m=====
70 很好！我已经更新了您的信息：
71 - 姓名：韩立
72 - 年龄：200岁
73 - 爱好：修仙、跑步
74 作为一位元婴期的前辈，您一定有着丰富的修炼经验和人生智慧。跑步这个爱好很有意思，对于修
75 仙者来说，这既是锻炼身体的好方法，也能让心境更加平和。您平时是在宗门内跑步，还是在山林
76 间奔跑呢？
77 ===== -> thread_2 messages <-
78 =====
79 =====•[1m Human Message
80 •[0m=====
81 你还记得我吗？
82 =====•[1m Ai Message
83 •[0m=====

```

```

80
81 我来查看一下之前记录的用户信息。
82 Tool Calls:
83   read_user_info (call_00_ijgxe0ZKvXCNYkRjX3EFGp8q)
84   Call ID: call_00_ijgxe0ZKvXCNYkRjX3EFGp8q
85   Args:
86   =====[1m Tool Message
87   •[0m=====
88   Name: read_user_info
89   username='韩立' age=200 hobbies=['修仙', '跑步']
90   =====[1m Ai Message
91   •[0m=====
92 当然记得！你是韩立，200岁，喜欢修仙和跑步。很高兴再次见到你！最近修仙进展如何？还是经常跑步锻炼吗？

```

分析

1. 前两次invoke通过相同的thread_id串联为一个会话，共享短期记忆
2. context是运行时静态配置，每次invoke互相独立
3. 长期记忆可以在任意位置访问，全局共享同一份信息
4. tool参数列表中的ToolRuntime是Langchain / Langgraph运行时注入的，在工具schema中没有体现
5. ToolRuntime的默认泛型是[None, Any]，第一个是Context的泛型，第二个是State对象的泛型。如果不显式指定上下文对象的类型，则Langchain/Langgraph底层会认为上下文对象为None，invoke调用时传递了context就会抛出警告，所以这里将UserContext作为ToolRuntime的第一个泛型传递。